LÖVE WIKI · PART 8 OF 9

Tutorials

30 pages

wiki pages 2057–2086

Contents of this part

Tutorial:A Guide to Getting Started with Love2D
Tutorial:Advanced Tiled Loader
Tutorial:Animation
Tutorial:Audio
Tutorial:Baseline 2D Platformer
Tutorial:Callback Functions
Tutorial:Cameras
Tutorial:Cameras:camera
Tutorial:Custom Cursors
Tutorial:Draw Origins
Tutorial:Drawing Order
Tutorial:Efficient Tile-based Scrolling
Tutorial:Fine Tile-based Scrolling
Tutorial:Fire Toward Mouse
Tutorial:Fonts and Text
Tutorial:Graphic Transformations
Tutorial:Gridlocked Player
Tutorial:Hamster Ball
Tutorial:Introduction to Shaders
Tutorial:Isometric Graphics
Tutorial:Networking with UDP
Tutorial:Networking with UDP-TheClient
Tutorial:Networking with UDP-TheServer
Tutorial:Object Follows Another Object
Tutorial:Object Follows Mouse
Tutorial:Physics
Tutorial:PhysicsCollisionCallbacks
Tutorial:PhysicsDrawing
Tutorial:Tile-based Scrolling
Tutorial:Using Input

Tutorial:A Guide to Getting Started with Love2D

Tutorial:A Guide to Getting Started with Love2D · revision 14498 · last edited 2021-03-14

Tutorial:Advanced Tiled Loader

Tutorial:Advanced Tiled Loader · revision 18930 · last edited 2023-07-05

In this tutorial we will cover the basics of using Kadoba's Advanced Tiled Loader. You'll need the Advanced Tiled Loader (in the AdvTiledLoader directory), a mapfile generated by Tiled, and a texturepack in the appropriate location. Let's start with the love.load function:

```
function love.load()

  loader = require("AdvTiledLoader.Loader")
  loader.path = "maps/" --Change this to wherever your .tmx files are
  map = loader.load("mapfile.tmx") --Change this to the name of your mapfile
  tx = 0
  ty = 0
  scale = 1 -- Adjust zoom with this

end
```

Here we load the ATL and tell it where our maps are. We load the maps and initialize our variables. Moving onto love.draw:

```
function love.draw()
  local ftx, fty = math.floor(tx), math.floor(ty)
  love.graphics.push()
  love.graphics.scale(scale)
  love.graphics.translate(ftx, fty)
  map:draw()
  love.graphics.pop()
end
```

In the love.draw function we change tx and ty into integers. This makes for slightly smoother graphics should. We move around the image with love.graphics.translate, and then we call map:draw to actually render it for us.

```
function love.update(dt)
  if love.keyboard.isDown("up") then ty = ty + 250*dt end
  if love.keyboard.isDown("down") then ty = ty - 250*dt end
  if love.keyboard.isDown("left") then tx = tx + 250*dt end
  if love.keyboard.isDown("right") then tx = tx - 250*dt end
end
```

Here we listen for a keypress and then adjust the translate variables, tx and ty appropriately.

More information can be found on Kadoba's original post here (http://love2d.org/forums/viewtopic.php?f=5&t=2567). More advanced samples are included with the loader itself.

See Also

Tile-based Scrolling
Efficient Tile-based Scrolling Tutorials

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Animation

Tutorial:Animation · revision 21722 · last edited 2023-07-05

On this page

- Loading animations
- Updating the animation
- Draw ing the animation
- Task
- Full Sourcecode
- Other Languages

Before we begin. This is a tutorial for semi advanced users. You are expected to know about tables, loops and the basics of drawing in Löve2D. And of course how to run a Löve2D game.

Alright. Let's get to it shall we?

I'm going to cover the basics of sprite based animation. This means you have a series of images which are displayed one after another. If you intent to create your own spritesheet make sure to leave at least 1px of pure transparency between the individual sprites. Otherwise you might see artifacts from the next or previous image and nobody wants that!

Loading animations

For this tutorial we'll be using a unnamed old hero. Generously made available for everyone by GrafxKid on OpenGameArt.org (https://opengameart.org/content/classic-hero). Download the image below and place it next to your main.lua in the folder.



oldHero.png

To be able to create multiple animations we want a reusable function that provides us with a table that we can use for everything related to the animation. We start this by defining a new function, storing our image in a variable and creating a new local table (the return value will be our table. We don't want some random animation table be globally defined or worse yet overwrite other data!)

The following parameters are going to be needed:

- image

An image object created with `love.graphics.newImage(filepath)`

- width

The width of each individual sprite. We are going to assume all sprites have the same size.

- height

The height of each individual sprite.

- duration

How long the animation takes before it loops back to the first frame.

```
function newAnimation(image, width, height, duration)
    local animation = {}
    animation.spriteSheet = image;
```

```
    return animation
end
```

You can try to just draw this directly with `love.graphics.draw(animation.spriteSheet)`. However. Then we'd just have all the images drawn next to each other. Certainly not what we want. Which is where Quads come in very handy!

They define a part of the image which will be drawn instead of the whole image. Exactly what we need!

Now we could define each quad individually. But that's going to be very annoying if we have several animations and hundreds of sprites! So instead we will load it within a loop that detects how many sprites are within an image. For this setup there must not be empty spaces and the image must contain exactly one animation.

```
function newAnimation(image, width, height, duration)
    local animation = {}
    animation.spriteSheet = image;
    animation.quads = {};

    for y = 0, image:getHeight() - height, height do
        for x = 0, image:getWidth() - width, width do
            table.insert(animation.quads, love.graphics.newQuad(x, y, width, height, image:getDimensions()))
        end
    end

    return animation
end
```

The 3 parameters to the for loop mean the following:

1. The starting value for y or x.

2. The maximum value (in this case the total width or height of the reference image)

3. How much should be added per step. In our case the size of the sprite.

With this we get the location on the sprite sheet from y and x!

You might wonder why we do "image:getHeight() - height" instead of just testing against the height of the image. This is because we want to make sure another sprite still fits on the sprite sheet. The sheet itself might not have the exact size we want it to. If it were one pixel too large. We do not want to add another quad (which would render as nothing) but instead ignore it.

Now we have 6 Quads ranging from index 1 - 6 in the table. Awesome!

But we have the issue that we have essentially 6 images that we can draw individually. But we need to draw them one after another over time. Also we don't just want to play this animation. We might want to change the speed at which it plays.

To cover this we add two more variables to our animation table. Duration (which we expect as parameter) and currentTime which is used to measure how much time has passed.

```
function newAnimation(image, width, height, duration)
    local animation = {}
    animation.spriteSheet = image;
    animation.quads = {};

    for y = 0, image:getHeight() - height, height do
        for x = 0, image:getWidth() - width, width do
            table.insert(animation.quads, love.graphics.newQuad(x, y, width, height, image:getDimensions()))
        end
```

```
    end

    animation.duration = duration or 1
    animation.currentTime = 0

    return animation
end
```

Which concludes our animation creation function already!

Updating the animation

Next up we need to load our animation table (call our newly created function) and update our current time.

```
function love.load()
    animation = newAnimation(love.graphics.newImage("oldHero.png"), 16, 18, 1)
end

function love.update(dt)
    animation.currentTime = animation.currentTime + dt
    if animation.currentTime >= animation.duration then
        animation.currentTime = animation.currentTime - animation.duration
    end
end
```

The demo sprite sheet has a sprite size of 16 pixels wide and 18 pixels high. We intent to play all images over the course of 1 second.

In `love.update` we simply add dt (delta time aka the time since the last frame) to our current time. We now count upwards continuously!

But we will use the current time to determine which frame should be shown. As such we will want it to be between 0 and the value of "duration". The if simply checks if "currentTime" is more than "duration" in which case we subtract "duration". You could just set "currentTime" to 0 instead of subtracting "duration". However this will result in fractions of a second being lost every time the animation finishes and therefore slow down the animation playtime ever so slightly. Which can easily be avoided and should be avoided!

Now for the really interesting part!

Drawing the animation

How do we draw this?

Well. We have the duration and current time. With this info we can calculate a percentage! How much of the animation has passed so far?

If you've followed this tutorial correctly so far *"currentTime / duration"* will provide you with a number between 0 and 1. Which represents the percentage. 0.25 means 25% of the animation has passed.

Considering this we can search for the correct image to use! Since we already have a number between 0 and 1 we can simply multiply this percentage with our total amount of images and get a number between 0 and 6!

*currentTime / duration * #quads*

However. If we try to get this from our table we will run into the issue that this is not a whole number. But our images are stored with whole numbers! So attempting to get the image at index "4.75" will give us nothing. Bummer!

Fear not. The solution is not too difficult.

"currentTime" will be a number between 0 and just below "duration" (because we reduce "currentTime" if it is larger or *equal* "duration"). Which would result in a number between 0 and 5.

To transform this value from our decimal point value to a whole number between 1 and 6 we do the following:

```
math.floor(currentTime / duration * #quads) + 1
```

"math.floor" provides us with the next lower number. Which means in our case a number between 0 and 5. We add one pushing it to a number between 1 and 6. All the sprites we have!

Lövely!

Alright. So all that's left is to draw the appropriate quad!

This simply requires us to provide `love.graphics.draw` with the image reference (our spriteSheet) and the quad we want to use. Simple enough!

```
    local spriteNum = math.floor(animation.currentTime / animation.duration * #animation.quads) + 1
    love.graphics.draw(animation.spriteSheet, animation.quads[spriteNum])
```

And we are done! You should have a walking dude in the top left corner of your window when you execute this code!

You can change where and how it is drawn by providing more parameters to the draw function.

Disclaimer: This code is not game ready! It is meant to explain the basic logic behind animations. If you are searching for game ready code take a look at the list of Libraries available!

Task

To improve this code. Try to rewrite the update and draw function to be able to handle multiple animations.

You can load the same animation multiple times and store them in a special table to get started right away!

As hint as to how it could be done: You could put the drawing logic in the animation table as well. And build it so you can call "animation:draw(x, y, r, sx, sy, [...])" to draw it on the screen.

Good luck and have fun!

Full Sourcecode

```
function love.load()
    animation = newAnimation(love.graphics.newImage("oldHero.png"), 16, 18, 1)
end

function love.update(dt)
    animation.currentTime = animation.currentTime + dt
    if animation.currentTime >= animation.duration then
        animation.currentTime = animation.currentTime - animation.duration
    end
end

function love.draw()
    local spriteNum = math.floor(animation.currentTime / animation.duration * #animation.quads) + 1
    love.graphics.draw(animation.spriteSheet, animation.quads[spriteNum], 0, 0, 0, 4)
```

```
end

function newAnimation(image, width, height, duration)
    local animation = {}
    animation.spriteSheet = image;
    animation.quads = {};

    for y = 0, image:getHeight() - height, height do
        for x = 0, image:getWidth() - width, width do
            table.insert(animation.quads, love.graphics.newQuad(x, y, width, height, image:getDimensions()))
        end
    end

    animation.duration = duration or 1
    animation.currentTime = 0

    return animation
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Audio

Tutorial:Audio · revision 28082 · last edited 2025-09-30

On this page

- Formats
- Static vs. Streaming
- Audio control
- Example
- See Also
- Other Languages

In LÖVE, audio is the domain of the love.audio module, which uses OpenAL for playback. love.audio has only one type of audio object - a Source. You can load audio and play it like so:

```
sound = love.audio.newSource("pling.wav", "static") -- the "static" tells LÖVE to load the file into memory, good for short sound effects
music = love.audio.newSource("techno.ogg", "stream") -- the "stream" tells LÖVE to stream the file from disk, good for longer music tracks
sound:play()
music:play()
```

Formats

LÖVE supports a lot of audio formats, thanks to the love.sound module, which handles all the decoding. Supported formats include:

- MP3
- Ogg Vorbis
- WAVE
- and just about every tracker format you can think of - XM, MOD, and over twenty others (http://sourceforge.net/p/modplug-xmms/git/ci/master/tree/libmodplug/README#l39).

Ogg Vorbis and 16-bit WAVE are the recommended formats. Others may have minor quirks. For example, the MP3 decoder may pad a few samples depending on what encoder was used. These issues come from the libraries LÖVE uses to decode the audio files and can't be fixed in LÖVE directly.

Static vs. Streaming

Keep in mind that, if you pass `love.audio.newSource` "static" as a second argument, the sound file will be expanded into memory, so if you load a 5MB compressed .ogg file that way, it would consume ~50MB RAM when fully decompressed. Consider not using "static" in such cases.

If you omit the "static", the audio will be streamed from the file as it's played, something that saves a lot of memory when you're dealing with massive files.

Audio control

To pause, stop, change volume, looping, pitch, etc., simply call the relevant method of a Source.

```
src1 = love.audio.newSource("bang.wav", "static")
src2 = love.audio.newSource("bgm.mp3", "stream")

src1:setVolume(0.9) -- 90% of ordinary volume
src1:setPitch(0.5) -- one octave lower
src2:setVolume(0.7)
```

```
src1:play()
src2:play()
```

Example

```
function love.load()
    source = love.audio.newSource("Ye-Olde-Pub_Looping.mp3", "stream")
end

function love.update(dt)
    if not source:isPlaying( ) then
        love.audio.play( source )
    end
end
```

Further details can be found in the Source documentation.

See Also

- love.audio
- TEsound - A sound manager which makes it easier to use sound effects and music
- Minimalist Sound Manager - A snippet that makes usage of love.audio easier and automatically manages sources

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Baseline 2D Platformer

Tutorial:Baseline 2D Platformer · revision 30473 · last edited 2025-12-02

On this page

Overview
Part 1: The Platform
Part 2: The Player
Part 3: Player Movement
Part 4: Jumping
Part 5: The Whole Code

Overview

Part 1: The Platform
Part 2: The Player
Part 3: Player Movement
Part 4: Jumping
Part 5: The Whole Code

In this tutorial, we will be creating a "Baseline 2D Platformer". In this context, "baseline" means the basic functions. Therefore, in a baseline 2d platformer, you can see or experience the following:

- The platform for the player to move around on.
- The player avatar/character.
- The player's ability to move left and right.
- The player's ability to jump around.

Part 1: The Platform

First, let's create the platform. For tutorial purposes, the platform is a white rectangle that will encompass the bottom-half part of the game window.

```
platform = {}

function love.load()
        -- This is the height and the width of the platform.
    platform.width = love.graphics.getWidth()    -- This makes the platform as wide as the whole game window.
    platform.height = love.graphics.getHeight()  -- This makes the platform as tall as the whole game window.

        -- This is the coordinates where the platform will be rendered.
    platform.x = 0                               -- This starts drawing the platform at the left edge of the game window.
    platform.y = platform.height / 2             -- This starts drawing the platform at the very middle of the game window
end

function love.update(dt)

end

function love.draw()
    love.graphics.setColor(1, 1, 1)        -- This sets the platform color to white.

        -- The platform will now be drawn as a white rectangle while taking in the variables we declared above.
    love.graphics.rectangle('fill', platform.x, platform.y, platform.width, platform.height)
end
```

So for will have something that looks like this:



Nice! Now we have a simple white platform!

Part 2: The Player

Next will be the player character. For tutorial purposes, the player character will be a 32x32 purple block. This time, instead of using the love.graphics.rectangle, we will use love.graphics.draw. The reason for this is because the purple block would be an external file. This way, you can learn how to draw an image "internally" as well as how to draw an "external" image. You can get the purple block here.

> Note: Just as a quick reference, love.graphics.rectangle lets you draw a rectangle in the game window. We can consider this as a form of "internal-file rendering" (not the exact or proper term) since we are declaring its (possible) parameters in the game's code. On the other hand, love.graphics.draw lets you draw an image in the window - as long as the file type is supported. We can consider this as a form of "external-file rendering" (again, not the exact or proper term) since the image we are rendering is an external file being called by the game's code.

```
...
player = {}  -- Add this below the platform variable

function love.load()
    ...
        -- Add this below the platform variables.

        -- This is the coordinates where the player character will be rendered.
    player.x = love.graphics.getWidth() / 2   -- This sets the player at the middle of the screen based on the width of the game window.
    player.y = love.graphics.getHeight() / 2  -- This sets the player at the middle of the screen based on the height of the game window.

        -- This calls the file named "purple.png" and puts it in the variable called player.img.
    player.img = love.graphics.newImage('purple.png')
end

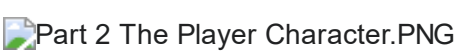
function love.update(dt)

end

function love.draw()
    ...
        -- Add this below the love.graphics.rectangle line.

        -- This draws the player.
    love.graphics.draw(player.img, player.x, player.y, 0, 1, 1, 0, 32)
end
```

Now we have something that looks like this:



Alright! Now the player exists!

Part 3: Player Movement

We have the platform, we have the player! Time to make this guy move. Remember that in 2d platformers, the player **must be able to move left and right!** We can't just have the player move forward to the right forever. To do this, we need to declare the speed the of the player's movement as well as assign keyboard inputs so that when the player presses a certain button, the character will move left or right depending on what is coded.

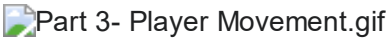
```
function love.load()
    ...
    player.speed = 200    -- This is the player's speed. This value can be change based on your liking.
end

function love.update(dt)
        -- This is how to assign keyboard inputs.

    if love.keyboard.isDown('d') then                    -- When the player presses and holds down the "D"
button:
        player.x = player.x + (player.speed * dt)    -- The player moves to the right.
    elseif love.keyboard.isDown('a') then                -- When the player presses and holds down the "A"
button:
        player.x = player.x - (player.speed * dt)    -- The player moves to the left.
    end
end
```

If you run the code, the player can now move left and right! **HOWEVER!** There is a problem. If you haven't noticed by now, if you keep on moving the character to the right (or to the left), it will go pass the game window and be out of the player's vision. For tutorial purposes, we don't want that. So inside the love.update() function:

```
function love.update(dt)
    if love.keyboard.isDown('d') then
        -- This makes sure that the character doesn't go pass the game window's right edge.
        if player.x < (love.graphics.getWidth() - player.img:getWidth()) then
            player.x = player.x + (player.speed * dt)
        end
    elseif love.keyboard.isDown('a') then
        -- This makes sure that the character doesn't go pass the game window's left edge.
        if player.x > 0 then
            player.x = player.x - (player.speed * dt)
        end
    end
end
```


Part 3- Player Movement.gif

Now the player can't get pass the screen. This way the player character will always be within the constraints of the game window.

Part 4: Jumping

All that's left now is jumping. You can't have a 2d platformer without jumping. Without it, player's can't really get over obstacles like pits or walls. It's time for some basic physics. For something to jump and fall, an object (in this case, the player's character) needs to have a Y-Axis Velocity, a Jump Height, and Gravity. For tutorial purposes, we will not be taking into consideration an object's mass however it is worth mentioning that giving

mass to an object can change the way it's physics works. First, let's declare the three things we've mentioned earlier. I will also be adding a variable called "ground". This is to indicate where the ground is. Think of it as the place where the feet should touch and land after jumping.

```
function love.load()
    ...
        -- Add this below the player.img
    player.ground = player.y      -- This makes the character land on the plaform.

    player.y_velocity = 0         -- Whenever the character hasn't jumped yet, the Y-Axis velocity is always at 0.

    player.jump_height = -300     -- Whenever the character jumps, he can reach this height.
    player.gravity = -500         -- Whenever the character falls, he will descend at this rate.
end
```

> Note: Feel free to change the values of player.jump_height and player.gravity to your liking. Just remember that for the most part, unless you want the game's physics to not work the usual way, player.gravity > player.jump_height

After declaring the variables, we will now proceed to making the character jump. To do this, we need to assign a key that will make the character jump.

```
...
function love.update(dt)
    ...
        -- Add below the right key assignment.

        -- This is in charge of player jumping.
    if love.keyboard.isDown('space') then                    -- Whenever the player presses or holds down the
Spacebar:

                -- The game checks if the player is on the ground. Remember that when the player is on the
ground, Y-Axis Velocity = 0.
        if player.y_velocity == 0 then
            player.y_velocity = player.jump_height    -- The player's Y-Axis Velocity is set to it's Jump Height.
        end
    end
end
```

We aren't done yet. If you try to jump, the character won't jump yet. This is because we haven't added the physics of the jump yet.

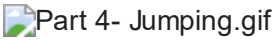
```
...
function love.update(dt)
    ...
    -- Add below the jump key assignment.

    -- This is in charge of the jump physics.
    if player.y_velocity ~= 0 then                                    -- The game checks if player has "jumped"
and left the ground.
        player.y = player.y + player.y_velocity * dt                -- This makes the character ascend/jump.
        player.y_velocity = player.y_velocity - player.gravity * dt -- This applies the gravity to the character.
    end

    -- This is in charge of collision, making sure that the character lands on the ground.
    if player.y > player.ground then    -- The game checks if the player has jumped.
        player.y_velocity = 0       -- The Y-Axis Velocity is set back to 0 meaning the character is on the
```

```
ground again.
        player.y = player.ground    -- The Y-Axis Velocity is set back to 0 meaning the character is on the
ground again.
    end
end
```

And finally, the character can now jump!


Part 4- Jumping.gif

Part 5: The Whole Code

Congratulations! You have created a Baseline 2D Platformer! Now for reference, the whole code:

```
platform = {}
player = {}

function love.load()
    platform.width = love.graphics.getWidth()
    platform.height = love.graphics.getHeight()

    platform.x = 0
    platform.y = platform.height / 2

    player.x = love.graphics.getWidth() / 2
    player.y = love.graphics.getHeight() / 2

    player.speed = 200

    player.img = love.graphics.newImage('purple.png')

    player.ground = player.y

    player.y_velocity = 0

    player.jump_height = -300
    player.gravity = -500
end

function love.update(dt)
    if love.keyboard.isDown('d') then
        if player.x < (love.graphics.getWidth() - player.img:getWidth()) then
            player.x = player.x + (player.speed * dt)
        end
    elseif love.keyboard.isDown('a') then
        if player.x > 0 then
            player.x = player.x - (player.speed * dt)
        end
    end

    if love.keyboard.isDown('space') then
        if player.y_velocity == 0 then
            player.y_velocity = player.jump_height
        end
    end

    if player.y_velocity ~= 0 then
```

```
        player.y = player.y + player.y_velocity * dt
        player.y_velocity = player.y_velocity - player.gravity * dt
    end

    if player.y > player.ground then
        player.y_velocity = 0
        player.y = player.ground
    end
end

function love.draw()
    love.graphics.setColor(1, 1, 1)
    love.graphics.rectangle('fill', platform.x, platform.y, platform.width, platform.height)

    love.graphics.draw(player.img, player.x, player.y, 0, 1, 1, 0, 32)
end
```

I hope this tutorial helped you in any way and I thank you for reading and trying this tutorial.

Tutorial:Callback Functions

Tutorial:Callback Functions · revision 29373 · last edited 2024-11-14

On this page

- love.load
- love.update
- love.draw
- love.mousepressed
- love.mousereleased
- love.keypressed
- love.keyreleased
- love.focus
- love.quit
- Other languages

The callback functions in LÖVE are called by love.run to perform various tasks and are all optional. However, a fully-featured game experience would probably utilize nearly all of them, so it's wise to know what they are.

A callback, for those new to programming or otherwise unfamiliar with the term, is a function which works backwards in a sense. In a regular function like love.graphics.draw or math.floor, you call it and LÖVE or Lua does something. A callback, on the other hand, is a function that you code and LÖVE calls at certain times. This makes it easy to keep your code organized and optimal. For example, since love.load will only get called once when the game is first started (before any other callback), it's a fine place to put code which loads game content and otherwise prepares things.

love.load

```
function love.load()
   image = love.graphics.newImage("cake.jpg")
   love.graphics.setNewFont(12)
   love.graphics.setColor(0,0,0)
   love.graphics.setBackgroundColor(255,255,255)
end
```

This function gets called only once, when the game is started, and is usually where you would load resources, initialize variables and set specific settings. All those things can be done anywhere else as well, but doing them here means that they are done once only, saving a lot of system resources.

love.update

```
function love.update(dt)
   if love.keyboard.isDown("up") then
      num = num + 100 * dt -- this would increment num by 100 per second
   end
end
```

This function is called continuously and will probably be where most of your math is done. 'dt' stands for "delta time" and is the amount of seconds since the last time this function was called (which is usually a small value like 0.025714).

love.draw

```
function love.draw()
   love.graphics.draw(image, imgx, imgy)
   love.graphics.print("Click and drag the cake around or use the arrow keys", 10, 10)
end
```

`love.draw` is where all the drawing happens (if that wasn't obvious enough already) and if you call any of the `love.graphics.draw` outside of this function then it's not going to have any effect. This function is also called continuously so keep in mind that if you change the font/color/mode/etc at the end of the function then it will have an effect on things at the beginning of the function. For example:

```
function love.load()
   love.graphics.setColor(0,0,0)
end

function love.draw()
   love.graphics.print("This text is not black because of the line below", 100, 100)
   love.graphics.setColor(255,0,0)
   love.graphics.print("This text is red", 100, 200)
end
```

love.mousepressed

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

```
function love.mousepressed(x, y, button, istouch)
   if button == 1 then
      imgx = x -- move image to where mouse clicked
      imgy = y
   end
end
```

This function is called whenever a mouse button is pressed and it receives the button and the coordinates of where it was pressed. The button can be any of the button index that was pressed. This function goes very well along with `love.mousereleased`.

love.mousereleased

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

```
function love.mousereleased(x, y, button, istouch)
   if button == 1 then
      fireSlingshot(x,y) -- this totally awesome custom function is defined elsewhere
   end
end
```

This function is called whenever a mouse button is released and it receives the button and the coordinates of where it was released. You can have this function together with `love.mousepressed` or separate, they aren't connected in any way.

love.keypressed

```
function love.keypressed(key)
   if key == 'b' then
      text = "The B key was pressed."
   elseif key == 'a' then
      a_down = true
```

```
    end
end
```

This function is called whenever a keyboard key is pressed and receives the key that was pressed. The key can be any of the constants. This functions goes very well along with `love.keyreleased`.

love.keyreleased

```
function love.keyreleased(key)
   if key == 'b' then
      text = "The B key was released."
   elseif key == 'a' then
      a_down = false
   end
end
```

This function is called whenever a keyboard key is released and receives the key that was released. You can have this function together with `love.keypressed` or separate, they aren't connected in any way.

love.focus

```
function love.focus(f)
  if not f then
    print("LOST FOCUS")
  else
    print("GAINED FOCUS")
  end
end
```

This function is called whenever the user clicks off and on the LÖVE window. For instance, if they are playing a windowed game and a user clicks on their Internet browser, the game could be notified and automatically pause the game.

```
function love.focus(f) gameIsPaused = not f end

function love.update(dt)
    if gameIsPaused then return end

    -- The rest of your love.update code goes here
end
```

love.quit

```
function love.quit()
  print("Thanks for playing! Come back soon!")
end
```

This function is called whenever the user clicks the window's close button (often an X). For instance, if the user decides they are done playing, they could click the close button. Then, before it closes, the game can save its state.

Those are the callback functions and their basic usage.

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Cameras

Tutorial:Cameras · revision 14500 · last edited 2023-07-05

BlackBulletIV has written a series on creating cameras in LÖVE entitled "Cameras in Love2D". It's split into three parts, which you can read by clicking on the links:

- Part 1: The Basics (http://nova-fusion.com/2011/04/19/cameras-in-love2d-part-1-the-basics/)
- Part 2: Parallax Scrolling (http://nova-fusion.com/2011/04/22/cameras-in-love2d-part-2-parallax-scrolling/)
- Part 3: Movement Bounds (http://nova-fusion.com/2011/05/09/cameras-in-love2d-part-3-movement-bounds/)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Cameras:camera

Tutorial:Cameras:camera · revision 12034 · last edited 2023-07-05

Place for source code for camera module.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Custom Cursors

Tutorial:Custom Cursors · revision 26789 · last edited 2021-07-20


Please note, that you can use love.mouse.setCursor since 0.9.0

BlackBulletIV's tutorial on customizing the mouse cursor:

Custom Cursors in Love2D (http://web.archive.org/web/20201028203204/http://nova-fusion.com/2012/09/20/custom-cursors-in-love2d/)

Tutorial:Draw Origins

Tutorial:Draw Origins · revision 14499 · last edited 2016-11-11

Tutorial:Drawing Order

Tutorial:Drawing Order · revision 12296 · last edited 2023-07-05

As LÖVE doesn't have a built-in z-ordering system, you may encounter some issues when trying to create a game that automatically draws a character in front or behind objects depending on their respective positions such as is customarily found in adventure games. This tutorial will show you how to do this in a very simplistic manner.

Note: In this tutorial, when I refer to an object being "in front" of the character I mean that it is between the character and the screen (ie: you). "Behind" means that the character is between the screen and the object.

First, we need some resources:

```
-- the background for our scene
scene = love.graphics.newImage("room.png")
-- the character we will be moving around
person = love.graphics.newImage("guy.png")
-- an object to move around
object = love.graphics.newImage("ball.png")
```

Alright, now that we have our resources we need some variables to contain the positions of our person and objects:

```
-- the character position
character = {400,400} -- x,y

-- a bunch of objects, each with a position
objects = {}
objects[1] = {550,370}
objects[2] = {220,390}
objects[3] = {600,410}
objects[4] = {300,450}
objects[5] = {400,530}
```

Make sure that your list of objects are ordered by their y-position. This becomes important when drawing them. For a general application, one can first use the Lua function `table.sort`.

```
function orderY(a,b)
  return a[2] < b[2]
end
table.sort(objects, orderY)
```

Although, You must be careful not to store the index to an object directly. As the objects index will change after the sort (You may wish to make a copy for drawing purposes only)

To draw these objects, we are going to use

```
love.graphics.draw(object, objects[i][1] - object:getWidth()/2, objects[i][2] -  object:getHeight())
```

The `object` parameter specifies the image we're going to draw and the next two arguments specify where the upper left corner of the image should be drawn on the screen. We subtract by half of the width and the height of the image to place the image's bottom horizontal center at `object[i]`'s specified x and y coordinate.

Ok, now we are ready to draw the whole shebang, assuming that you have created some way of moving your character around the screen (or else this is just pointless as you won't see the difference). If you don't know how to do that, I suggest looking at some of our other tutorials for assistance. For simplicity I am just going to

show you the entire `love.draw` function and then explain what happens:

```
function love.draw()
    love.graphics.draw(scene, love.graphics:getWidth() / 2 - scene:getWidth()/2,
        love.graphics:getHeight() / 2 - scene:getHeight() / 2) -- draw at the center of the screen

    local drawn = false -- true when the character has been drawn

    for i,v in ipairs(objects) do
        if not drawn and objects[i][2] > character[2] then
            love.graphics.draw(person, character[1] - person:getWidth()/2, character[2] - person:getHeight())
            drawn = true
        end
    love.graphics.draw(object, objects[i][1] - object:getWidth()/2, objects[i][2] - object:getHeight())
   end

   if not drawn then -- if the person is below all objects it won't be drawn within the for loop
      love.graphics.draw(person, character[1] - person:getWidth()/2, character[2] - person:getHeight())
   end

   -- any foreground objects go here

end
```

First of all we draw our scene as it normally is in the back (like any potential foreground objects will always be in the front). Then I create a variable called `drawn`. This will tell us if the character has been drawn to ensure that he isn't re-drawn multiple times which will cause him to be in front of objects where he shouldn't be. Next I go into a for loop and iterate through all of our objects (which are ordered from furthest away to closest). Then I check if the object is supposed to be in front of our character, if it is then you should draw the character (and set the drawn variable). The reason for this is because this for loop will go through and draw the objects (which are all the same image in this case, but that doesn't have to be so) until it comes to an object which should be in front of the chracter which is when it draws the character and then continues to draw the rest of the objects. If it comes to the end of the for loop and hasn't drawn the character yet it means that the character is in front of all the objects so it draws it at the end.

That's all. Download the demo and give it a go. It may be very simple but this tutorial is only about drawing order and if you want to know how to check if the character is colliding with the object or you want the character to go where you have clicked then you're on your own (until a tutorial is made for that). If this is not straightforward enough, please feel free to comment on it on our forum (http://www.love2d.org/forums/).

See Also

An alternative, scalable implementation : Skip list:Drawing Order

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Efficient Tile-based Scrolling

Tutorial:Efficient Tile-based Scrolling · revision 17598 · last edited 2023-07-05

On this page

- Quads and SpriteBatch
- Map Initialization
- Adding a SpriteBatch as Tilemap
- Discrete Moving
- Continuous Movement
- Putting It All Together
- Other languages

This tutorial introduces the SpriteBatch class for more efficient tile-based scrolling. For a tutorial on basic tile-based scrolling, see Tutorial:Tile-based_Scrolling.

Quads and SpriteBatch



The love.graphics.draw method can draw a portion of an image specified by a Quad. If quads are drawn from different parts of a single image, we can make the system more efficient by using a SpriteBatch and not changing the quads every frame.

When creating a SpriteBatch, we must specify the image that we take the quads ("tiles") from and the maximum number of quads that we will be adding. In this case, the maximum number of tiles visible on the screen.

```
tilesetBatch = love.graphics.newSpriteBatch(tilesetImage, tilesDisplayWidth * tilesDisplayHeight)
```

Screenshot of tile-based scrolling with a simple map generator.

Map Initialization

We initialize a map with the following. See the Tile-based Scrolling Tutorial for an explanation.

```
function love.load()
  mapWidth = 60
  mapHeight = 40

  map = {}
  for x=1,mapWidth do
    map[x] = {}
    for y=1,mapHeight do
      map[x][y] = love.math.random(0,3)
    end
  end

  mapX = 1
  mapY = 1
  tilesDisplayWidth = 26
  tilesDisplayHeight = 20

  zoomX = 1
```

```
  zoomY = 1
end
```

Adding a SpriteBatch as Tilemap

Next, we load a tileset, create quads for the tiles we want to use, and make a SpriteBatch object to hold the tiles. As an example, we are going to use a free tileset from http://silveiraneto.net/... (http://silveiraneto.net/2009/02/02/my-free-tileset-version-7/).



Example tileset.

```
function love.load()
  ... --map init

  tilesetImage = love.graphics.newImage( "tileset.png" )
  tilesetImage:setFilter("nearest", "linear") -- this
"linear filter" removes some artifacts if we were to scale
the tiles
  tileSize = 32

  -- grass
  tileQuads[0] = love.graphics.newQuad(0 * tileSize, 20 *
tileSize, tileSize, tileSize,
    tilesetImage:getWidth(), tilesetImage:getHeight())
  -- kitchen floor tile
  tileQuads[1] = love.graphics.newQuad(2 * tileSize, 0 *
tileSize, tileSize, tileSize,
    tilesetImage:getWidth(), tilesetImage:getHeight())
  -- parquet flooring
  tileQuads[2] = love.graphics.newQuad(4 * tileSize, 0 *
tileSize, tileSize, tileSize,
    tilesetImage:getWidth(), tilesetImage:getHeight())
  -- middle of red carpet
  tileQuads[3] = love.graphics.newQuad(3 * tileSize, 9 *
tileSize, tileSize, tileSize,
    tilesetImage:getWidth(), tilesetImage:getHeight())

  tilesetBatch = love.graphics.newSpriteBatch(tilesetImage, tilesDisplayWidth * tilesDisplayHeight)
end
```

We only wish to add to the SpriteBatch the tiles that are presently visible. To do this, we make a function that updates the tileset and call it whenever the map focus changes. We also call it once in the initialization.

```
function updateTilesetBatch()
  tilesetBatch:clear()
  for x=0, tilesDisplayWidth-1 do
    for y=0, tilesDisplayHeight-1 do
      tilesetBatch:add(tileQuads[map[x+mapX][y+mapY]], x*tileSize, y*tileSize)
    end
  end
  tilesetBatch:flush()
end
```

Finally, to draw the SpriteBatch, we just send it to `love.graphics.draw`.

```
function love.draw()
  love.graphics.draw(tilesetBatch)
end
```

Discrete Moving

Moving around the map is done the same way as in the Tile-based Scrolling tutorial; we check if any keys are pressed and update the map accordingly. We must also remember to update the SpriteBatch.

```
-- central function for moving the map
function moveMap(dx, dy)
  oldMapX = mapX
  oldMapY = mapY
  mapX = math.max(math.min(mapX + dx, mapWidth - tilesDisplayWidth), 1)
  mapY = math.max(math.min(mapY + dy, mapHeight - tilesDisplayHeight), 1)
  -- only update if we actually moved
  if math.floor(mapX) ~= math.floor(oldMapX) or math.floor(mapY) ~= math.floor(oldMapY) then
    updateTilesetBatch()
  end
end

function love.keypressed(key)
  if key == "up" then
    moveMap(0, -1)
  end
  if key == "down" then
    moveMap(0, 1)
  end
  if key == "left" then
    moveMap(-1, 0)
  end
  if key == "right" then
    moveMap(1, 0)
  end
end
```

Continuous Movement

We make the movement a bit nicer by allowing mapX and mapY to take on non-integer values. When adding quads to the SpriteBatch, we will only consider the integer part while the drawing will shift the SpriteBatch to handle the fractional part. We replace the love.keypressed callback with a love.update callback and move the map in small steps.

```
function love.update(dt)
  if love.keyboard.isDown("up")  then
    moveMap(0, -0.2 * tileSize * dt)
  end
  if love.keyboard.isDown("down")  then
    moveMap(0, 0.2 * tileSize * dt)
  end
  if love.keyboard.isDown("left")  then
    moveMap(-0.2 * tileSize * dt, 0)
  end
  if love.keyboard.isDown("right")  then
    moveMap(0.2 * tileSize * dt, 0)
```

```
  end
end
```

We add a floor to the SpriteBatch update.

```
function updateTilesetBatch()
  tilesetBatch:clear()
  for x=0, tilesDisplayWidth-1 do
    for y=0, tilesDisplayHeight-1 do
      tilesetBatch:add(tileQuads[map[x+math.floor(mapX)][y+math.floor(mapY)]],
        x*tileSize/2, y*tileSize/2)
    end
  end
  tilesetBatch:flush()
end
```

Finally, we shift the SpriteBatch by the fractional part.

```
function love.draw()
  love.graphics.draw(tilesetBatch,
    math.floor(-(mapX%1)*tileSize), math.floor(-(mapY%1)*tileSize))
  love.graphics.print("FPS: "..love.timer.getFPS(), 10, 20)
end
```

Putting It All Together

We have also added zoomX and zoomY variables to this code to allow, e.g., 16x16 tiles to be drawn as 32x32.

```
local map -- stores tiledata
local mapWidth, mapHeight -- width and height in tiles

local mapX, mapY -- view x,y in tiles. can be a fractional value like 3.25.

local tilesDisplayWidth, tilesDisplayHeight -- number of tiles to show
local zoomX, zoomY

local tilesetImage
local tileSize -- size of tiles in pixels
local tileQuads = {} -- parts of the tileset used for different tiles
local tilesetSprite

function love.load()
  setupMap()
  setupMapView()
  setupTileset()
  love.graphics.setFont(12)
end

function setupMap()
  mapWidth = 60
  mapHeight = 40

  map = {}
  for x=1,mapWidth do
    map[x] = {}
    for y=1,mapHeight do
```

```
      map[x][y] = love.math.random(0,3)
    end
  end
end

function setupMapView()
  mapX = 1
  mapY = 1
  tilesDisplayWidth = 26
  tilesDisplayHeight = 20

  zoomX = 1
  zoomY = 1
end

function setupTileset()
  tilesetImage = love.graphics.newImage( "tileset.png" )
  tilesetImage:setFilter("nearest", "linear") -- this "linear filter" removes some artifacts if we were to scale
the tiles
  tileSize = 32

  -- grass
  tileQuads[0] = love.graphics.newQuad(0 * tileSize, 20 * tileSize, tileSize, tileSize,
    tilesetImage:getWidth(), tilesetImage:getHeight())
  -- kitchen floor tile
  tileQuads[1] = love.graphics.newQuad(2 * tileSize, 0 * tileSize, tileSize, tileSize,
    tilesetImage:getWidth(), tilesetImage:getHeight())
  -- parquet flooring
  tileQuads[2] = love.graphics.newQuad(4 * tileSize, 0 * tileSize, tileSize, tileSize,
    tilesetImage:getWidth(), tilesetImage:getHeight())
  -- middle of red carpet
  tileQuads[3] = love.graphics.newQuad(3 * tileSize, 9 * tileSize, tileSize, tileSize,
    tilesetImage:getWidth(), tilesetImage:getHeight())

  tilesetBatch = love.graphics.newSpriteBatch(tilesetImage, tilesDisplayWidth * tilesDisplayHeight)

  updateTilesetBatch()
end

function updateTilesetBatch()
  tilesetBatch:clear()
  for x=0, tilesDisplayWidth-1 do
    for y=0, tilesDisplayHeight-1 do
      tilesetBatch:add(tileQuads[map[x+math.floor(mapX)][y+math.floor(mapY)]],
        x*tileSize, y*tileSize)
    end
  end
  tilesetBatch:flush()
end

-- central function for moving the map
function moveMap(dx, dy)
  oldMapX = mapX
  oldMapY = mapY
  mapX = math.max(math.min(mapX + dx, mapWidth - tilesDisplayWidth), 1)
  mapY = math.max(math.min(mapY + dy, mapHeight - tilesDisplayHeight), 1)
  -- only update if we actually moved
  if math.floor(mapX) ~= math.floor(oldMapX) or math.floor(mapY) ~= math.floor(oldMapY) then
    updateTilesetBatch()
```

```
  end
end

function love.update(dt)
  if love.keyboard.isDown("up")  then
    moveMap(0, -0.2 * tileSize * dt)
  end
  if love.keyboard.isDown("down")  then
    moveMap(0, 0.2 * tileSize * dt)
  end
  if love.keyboard.isDown("left")  then
    moveMap(-0.2 * tileSize * dt, 0)
  end
  if love.keyboard.isDown("right")  then
    moveMap(0.2 * tileSize * dt, 0)
  end
end

function love.draw()
  love.graphics.draw(tilesetBatch,
    math.floor(-zoomX*(mapX%1)*tileSize), math.floor(-zoomY*(mapY%1)*tileSize),
    0, zoomX, zoomY)
  love.graphics.print("FPS: "..love.timer.getFPS(), 10, 20)
end
```

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Fine Tile-based Scrolling

Tutorial:Fine Tile-based Scrolling · revision 11876 · last edited 2023-07-05

This is an expansion upon the code in Tile-based Scrolling. It assumes a tile size of 16x16 and a window size of 320x240.

```
function love.load()
    -- our tiles
    tile = {}
    for i=0,3 do -- change 3 to the number of tile images minus 1.
        tile[i] = love.graphics.newImage( "tile"..i..".png" )
    end

    -- the map (random junk + copy and paste)
    map={
    { 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 0, 0, 0, 0},
    { 3, 1, 0, 0, 2, 2, 2, 0, 3, 0, 3, 0, 1, 1, 1, 0, 0, 3, 0, 0, 0},
    { 3, 1, 0, 0, 2, 0, 2, 0, 3, 0, 3, 0, 1, 0, 0, 0, 0, 0, 3, 0, 0},
    { 3, 1, 1, 0, 2, 2, 2, 0, 0, 3, 0, 0, 1, 1, 0, 0, 0, 0, 0, 3, 0},
    { 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 3},
    { 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 2},
    { 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 2, 2, 2, 0, 3, 3, 3, 0, 1, 1, 1, 0, 2, 0, 0, 0, 0, 0, 0, 0},
    { 0, 2, 0, 0, 0, 3, 0, 3, 0, 1, 0, 1, 0, 2, 0, 0, 0, 0, 0, 0, 1},
    { 0, 2, 0, 0, 0, 3, 0, 3, 0, 1, 0, 1, 0, 2, 0, 0, 0, 0, 0, 0, 0},
    { 0, 2, 2, 2, 0, 3, 3, 3, 0, 1, 1, 1, 0, 2, 2, 2, 0, 0, 0, 0, 0},
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 1, 0, 0, 2, 2, 2, 0, 3, 0, 3, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0},
    { 0, 1, 0, 0, 2, 0, 2, 0, 3, 0, 3, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 1, 1, 0, 2, 2, 2, 0, 0, 3, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0},
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 3},
    { 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0},
    { 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    { 0, 1, 0, 0, 0, 1, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
    }

    -- map variables
    map_w = #map[1] -- Obtains the width of the first row of the map
    map_h = #map -- Obtains the height of the map
    map_x = 0
    map_y = 0
    map_display_buffer = 2 -- We have to buffer one tile before and behind our viewpoint.
                           -- Otherwise, the tiles will just pop into view, and we don't want that.
    map_display_w = 20
    map_display_h = 15
    tile_w = 16
    tile_h = 16
```

```
end

function draw_map()
    offset_x = map_x % tile_w
    offset_y = map_y % tile_h
    firstTile_x = math.floor(map_x / tile_w)
    firstTile_y = math.floor(map_y / tile_h)

    for y=1, (map_display_h + map_display_buffer) do
        for x=1, (map_display_w + map_display_buffer) do
            -- Note that this condition block allows us to go beyond the edge of the map.
            if y+firstTile_y >= 1 and y+firstTile_y <= map_h
                and x+firstTile_x >= 1 and x+firstTile_x <= map_w
            then
                love.graphics.draw(
                    tile[map[y+firstTile_y][x+firstTile_x]],
                    ((x-1)*tile_w) - offset_x - tile_w/2,
                    ((y-1)*tile_h) - offset_y - tile_h/2)
            end
        end
    end
end

function love.update( dt )
    local speed = 300 * dt
    -- get input
    if love.keyboard.isDown( "up" ) then
        map_y = map_y - speed
    end
    if love.keyboard.isDown( "down" ) then
        map_y = map_y + speed
    end

    if love.keyboard.isDown( "left" ) then
        map_x = map_x - speed
    end
    if love.keyboard.isDown( "right" ) then
        map_x = map_x + speed
    end
    if love.keyboard.isDown( "escape" ) then
        love.event.quit()
    end

    -- check boundaries. remove this section if you don't wish to be constrained to the map.
    if map_x < 0 then
        map_x = 0
    end

    if map_y < 0 then
        map_y = 0
    end

    if map_x > map_w * tile_w - map_display_w * tile_w - 1 then
        map_x = map_w * tile_w - map_display_w * tile_w - 1
    end

    if map_y > map_h * tile_h - map_display_h * tile_h - 1 then
        map_y = map_h * tile_h - map_display_h * tile_h - 1
    end
```

```
end

function love.draw()
    draw_map()
end
```

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Fire Toward Mouse

Tutorial:Fire Toward Mouse · revision 24127 · last edited 2023-07-05

On this page

- Initialization
- Drawing Everything
- Determining the Bullet's New Position
- Firing the Bullet
- Conclusion
- Other Languages

This tutorial describes how to make a bullet fire toward the mouse when the user clicks. It is assumed that you know the basics of LOVE and Lua. All code takes place within main.lua.

Initialization

```
function love.load()
    love.graphics.setBackgroundColor(0.21, 0.67, 0.97)

    bulletSpeed = 250

    bullets = {}
    player = {x=250, y=250, width=15, height=15}
end
```

The first line just sets the background color as a nice blue. bulletSpeed is our variable that defines how fast a bullet will travel in pixels per second. The table bullets will hold a list of all our bullets and the table player holds all the info about our player.

Each bullet will be its own table inside the bullets table. It will contain the properties x, y, dx, and dy. The dx and dy variables define how much the bullet should move in pixels per second on the x and y axis.

Drawing Everything

```
function love.draw()
    love.graphics.setColor(1, 1, 1)
    love.graphics.rectangle("fill",  player.x, player.y, player.width, player.height)

    love.graphics.setColor(0.5, 0.5, 0.5)
    for i,v in ipairs(bullets) do
        love.graphics.circle("fill", v.x, v.y, 3)
    end
end
```

The first thing we do in the draw function is to set the color as white. After that, we draw a rectangle (which will represent the player).

The next segment sets the color as gray for the bullet. The for statement will go through each value in our table of bullets. i is the index of the current bullet and v is its value. For each bullet we draw it as a circle on the screen

Determining the Bullet's New Position

```
function love.update(dt)
    for i,v in ipairs(bullets) do
```

```
        v.x = v.x + (v.dx * dt)
        v.y = v.y + (v.dy * dt)
    end
end
```

Once again we go through the list of bullets. For each one we update its x and y position. We do this by taking the current position and adding the change multiplied by the delta time.

Firing the Bullet

```
function love.mousepressed(x, y, button)
    if button == 1 then
        local startX = player.x + player.width / 2
        local startY = player.y + player.height / 2
        local mouseX = x
        local mouseY = y

        local angle = math.atan2((mouseY - startY), (mouseX - startX))

        local bulletDx = bulletSpeed * math.cos(angle)
        local bulletDy = bulletSpeed * math.sin(angle)

        table.insert(bullets, {x = startX, y = startY, dx = bulletDx, dy = bulletDy})
    end
end
```

The if statement at the beginning of this makes it so that we only fire a bullet if the user left clicks. The *startX* and *startY* variables define the origin of the bullet. We want the bullet to start at the center of the player so we use *player.x + player.width / 2* to get that.

To get angle we use *math.atan2* and pass it the difference in y and the difference in x. This function is very useful for finding the angle between any two objects.

We find *bulletDx* and *bulletDy* by using some more trig. These varibales will define how fast the bullet will move in the x and y plane.

The final line here inserts our new bullet into the list of bullets

Conclusion

When all put together you get the start of a top-down shooter game. Try out making the player movable and making the bullets disappear when they go off screen.

--Somethingmoreunique 03:22, 25 December 2011 (GMT)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Fonts and Text

Tutorial:Fonts and Text · revision 21474 · last edited 2023-07-05

On this page

- Creating a font object using the default font
- Creating a font object using a font file
- Creating a font object using an image
- Using the font object
- Drawing and formatting text
- Other languages

Adding and using fonts in LÖVE isn't very complicated, but what you have to keep in mind is that you need to set the font before you can use it. There is no way to render text directly from a font object, it must always be done via the `love.graphics.print` function which will render text based on the current font. There are three types of fonts available in love:

- The default font (Bitstream Vera Sans)
- Any font file supported by FreeType 2
- An image containing glyphs (ImageFont)

Creating a font object using the default font

The default font is available for everyone and anyone and can be created using the following code:

```
font = love.graphics.newFont(14) -- the number denotes the font size
```

Creating a font object using a font file

Font files are just normal files that you download from the internet (or create yourself) that contains font information. They can be used just like the default font, just that you specify the filepath along with the font size:

```
font = love.graphics.newFont("AwesomeFont.ttf", 15)
```

Keep in mind the that some fonts you download come with restrictions on when/how you can use them.

Creating a font object using an image

An image font is a little tougher to explain, as it all depends on a specifically formatted image. If you look at this example image you will see that it contains glyphs along one row, each separated by a specific color. This color is used to separate the glyphs and needs to be found at between every character and at the beginning of the list (all the way to the left). Creating a font based on this image would be as follows:

```
font = love.graphics.newImageFont("imagefont.png",
    " abcdefghijklmnopqrstuvwxyz" ..
    "ABCDEFGHIJKLMNOPQRSTUVWXYZ0" ..
    "123456789.,!?-+/():;%&`'*#=[]\"")
```

Warning: Make sure that the image starts and also ends with the separation color or else the last character (in this case ") won't work.

Using the font object

Now that you have created your font object, you need to set it as the current font before it can be used. That is done with the following code:

```
-- font = the font object you made before
love.graphics.setFont(font)
```

One can also immediately use a TTF-font by calling

```
love.graphics.setNewFont(filename, size)
```

Or, in the case of the default font,

```
love.graphics.setNewFont(size)
```

Drawing and formatting text

So now that you have your wonderful new font object, you want to use it for something. To draw text, you simply call the `love.graphics.print` function:

```
love.graphics.print("This is some awesome text", 100, 100)
```

You can easily draw numbers as well:

```
num = 15
love.graphics.print(num, 100, 150)
```

The default text rendering function can handle line breaks (the '\n' character) and will draw text aligned to the left:

```
love.graphics.print("This is the first line\nThis is the second one.\n(and a third)\n\nfifth", 100, 200)
```

The parameters '100' and '200' are the *x*- and *y*-locations of the text, respectively. In LÖVE versions prior to 0.6.0, `love.graphics.draw` was instead used to draw text.

However, LÖVE can also draw text with word wrapping and alignment by using the more complicated `love.graphics.printf` function:

```
love.graphics.printf("A really long sentence which will probably be broken into many pieces by my love.", 500,
100, 250, 'right')
```

Where, here, the parameters '500' and '100' are *x*- and *y*-locations, '250' is the maximum text width, and 'right' is the text alignment.

This way it is relatively easy to draw completely centered text if you pass the width of the window as the word-wrap limit and 0 as the x-position:

```
love.graphics.printf("This text will be centered no matter what.\n(and you LOVE it)", 0, 400, 800, 'center')
```

Text will always align to the left by default if you omit the alignment value:

```
love.graphics.printf("Aligned to the left, yes very much so.", 100, 500, 150)
```

That is how to draw text. For more information about the specific functions and their use, please visit the documentation.

Note: All fonts (even the image font) will be affected by the current color, there is no need to set the color mode to 'modulate'.

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Graphic Transformations

Tutorial:Graphic Transformations · revision 11753 · last edited 2023-07-05

This tutorial covers the basics of using graphic transformations.

love.graphics.push/love.graphics.pop

These two functions allow you to save and return to the previous transformation state. Every push must be paired up with a pop, and each pair can also be called within another push/pop.

Here's an example:

```
function love.draw()
    love.graphics.push() -- Store the previous transformation state
        -- Shift the coordinate system down 10, right 10
        love.graphics.translate(10,10)
        love.graphics.point(0,0)
    love.graphics.pop() -- Return to the previous transformation state
    love.graphics.point(0,0) -- The origin is back at (0,0)
end
```

Order of Transformations

Graphic transformations are not commutative. This means the call order affects the final result.

Here's an example:

```
function love.draw()
    love.graphics.push()
        love.graphics.translate(10,10)
        love.graphics.scale(-1,-1)
        love.graphics.point(10,10) -- The point is located at global (0,0)
    love.graphics.pop()

    love.graphics.scale(-1,-1)
    love.graphics.translate(10,10)
    love.graphics.point(10,10) -- The point is located at global (-20,-20)
end
```

There are two ways to visualize the above code.The first way is to think of each transformation as relative to the new coordinate system from previous transformations. For example, the scaling to the first point would be applied relative to the new origin at (10,10).

The second way (which is more intuitive in my humble opinion) is to visualize each transformation in REVERSE ORDER relative to the global coordinate system. Let's take the first point as an example. Scaling is applied first and places the new point at (-10,-10). Lastly, the translation moves the new point to (0,0).

See Also

- love.graphics.scale
- love.graphics.shear
- love.graphics.rotate

- love.graphics.translate

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Gridlocked Player

Tutorial:Gridlocked Player · revision 9628 · last edited 2021-03-14

In this tutorial, we will create a VERY basic game wherein you can move a "player" around the screen in fixed increments, but the player graphic moves in smaller increments. This could be useful if you are creating a game that is tile-based, and want your player to stay aligned to the grid, but don't want them to appear to teleport around.

Starting Code

To start with, we need to create a player. The player will have several aspects before we're done, so we're going to make the player variable a table inside of love.load().

```
function love.load()
    player = {}
end
```

Next we'll need to see what we're doing. We can render the player as a rectangle using love.graphics.rectangle() in love.draw().

```
function love.draw()
    love.graphics.rectangle("fill", 0, 0, 32, 32)
end
```

Let's fill in the player table a little so that the player has a position. Note that we're using the shorthand way of putting variables into a table: just define them inside the curly brackets, using commas to separate them. (We also need to change the parameters in love.graphics.rectangle to use these new values.)

```
function love.load()
    player = {
        x = 256,
        y = 256
    }
end

function love.draw()
    love.graphics.rectangle("fill", player.x, player.y, 32, 32)
end
```

 Grid player 1.gif

Now that the player has position, let's add in some controls using love.keypressed(). All we do is check to see if the key matches, and change the value in the player table.

```
function love.keypressed(key)
    if key == "down" then
        player.y = player.y + 32
    end
end
```

So we now have a little square that can teleport down by one grid unit (32 pixels in our case, the same length that we made our player).

Expansion

We can expand this to more keys using elseif. Notice that everything is still in units of 32, a common tile size that plays nice with PO2_Syndrome.

```
function love.keypressed(key)
    if key == "up" then
        player.y = player.y - 32
    elseif key == "down" then
        player.y = player.y + 32
    elseif key == "left" then
        player.x = player.x - 32
    elseif key == "right" then
        player.x = player.x + 32
    end
end
```

So we now have a happy little square that can jump around the screen as fast as you can hit the keyboard. Now let's add in the smooth transitions. To do this, we're going to need to shift up the variables in the player table. Don't forget that this will affect all references to the player table.

There are two things that we will now need. One, we need actual XY coordinates as opposed to the gridlocked coordinates that will now serve as destination coordinates. Two, we need to update the actual coordinates in love.update() to move towards the destination coordinates. We can do that with a simple math problem:

```
actual_x = actual_x - (actual_x - destination_x)
```

Or, in real Lua this time:

```
function love.load()
    player = {
        grid_x = 256,
        grid_y = 256,
        act_x = 200,
        act_y = 200
    }
end

function love.update(dt)
    player.act_y = player.act_y - (player.act_y - player.grid_y)
    player.act_x = player.act_x - (player.act_x - player.grid_x)
end

function love.draw()
    love.graphics.rectangle("fill", player.act_x, player.act_y, 32, 32)
end

function love.keypressed(key)
    if key == "up" then
        player.grid_y = player.grid_y - 32
    elseif key == "down" then
        player.grid_y = player.grid_y + 32
    elseif key == "left" then
        player.grid_x = player.grid_x - 32
```

```
    elseif key == "right" then
        player.grid_x = player.grid_x + 32
    end
end
```

As you probably noticed, you probably can't see any difference. We need to use love.update's dt parameter to control speed, by multiplying the difference of the positions by dt:

```
function love.update(dt)
    player.act_y = player.act_y - ((player.act_y - player.grid_y) * dt)
    player.act_x = player.act_x - ((player.act_x - player.grid_x) * dt)
end
```

Still not quite right though. Now the player is unbearably slow (but at least he's consistent from computer to computer!). Let's give the player a speed attribute to control how fast he moves, by multiplying dt by the speed:

```
function love.load()
    player = {
        grid_x = 256,
        grid_y = 256,
        act_x = 200,
        act_y = 200,
        speed = 10
    }
end

function love.update(dt)
    player.act_y = player.act_y - ((player.act_y - player.grid_y) * player.speed * dt)
    player.act_x = player.act_x - ((player.act_x - player.grid_x) * player.speed * dt)
end
```

Putting it all together

Here is the full code:

```
function love.load()
    player = {
        grid_x = 256,
        grid_y = 256,
        act_x = 200,
        act_y = 200,
        speed = 10
    }
end

function love.update(dt)
    player.act_y = player.act_y - ((player.act_y - player.grid_y) * player.speed * dt)
    player.act_x = player.act_x - ((player.act_x - player.grid_x) * player.speed * dt)
end

function love.draw()
    love.graphics.rectangle("fill", player.act_x, player.act_y, 32, 32)
end

function love.keypressed(key)
```

```
    if key == "up" then
        player.grid_y = player.grid_y - 32
    elseif key == "down" then
        player.grid_y = player.grid_y + 32
    elseif key == "left" then
        player.grid_x = player.grid_x - 32
    elseif key == "right" then
        player.grid_x = player.grid_x + 32
    end
end
```

Adding collision

Now let's add some world detail. To start with, we need to create a table of values that will dictate whether we can walk onto a grid or not.

```
function love.load()
map = {
    { 1, 1, 1, 1 },
    { 1, 0, 0, 1 },
    { 1, 0, 1, 1 },
    { 1, 1, 1, 1 }
}
end
```

This map table holds four subtables, each of which holds four values. Each subtable is a horizontal row in the map, with each value representing a single tile in that row. We used 1s and 0s because they're easy to type.

Now let's draw these values to the screen. Since we only have two types, we can get away with just drawing one of the types... let's say the 1s. We loop through the subtables by using a simple for loop, and then loop through the values inside each subtable.

```
function love.draw()
    for y=1, #map do
        for x=1, #map[y] do
            if map[y][x] == 1 then
                love.graphics.rectangle("line", x * 32, y * 32, 32, 32)
            end
        end
    end
end
```

A couple things to note about this double loop. The map table is arranged the way that we will see it. This means that are coordinates are flipped from how we usually think... instead of being x and then y, the map is y and then x. Also, the map is in units of 32 (one player width). That means that when we draw the unwalkable squares, we have to multiply their positions by 32.

 Grid player 2.gif

Now let's make a function to test if the player can walk over a spot on our map. This function will return true if we can walk over the spot, and false if we cannot. The function will be given two values: our change in x and our change in y.

```
function testMap(x, y)
    if map[(player.grid_y / 32) + y][(player.grid_x / 32) + x] == 1 then
        return false
    end
```

```
    return true
end
```

Things to note about this function. Since we're working in map units, the values passed in will usually just be 1, -1, or 0. That's why they are not divided by 32, unlike the player grid coordinates.

Speaking of the player grid coordinates, the reason we did not have to pass those in is because the player is a global variable.

So now we can put this function into use by sticking it in the keypressed function. Remember, the test function works in map units, rather than pixel units, so if we're trying to move up one grid, we pass (0, -1).

```
function love.keypressed(key)
    if key == "up" then
        if testMap(0, -1) then
            player.grid_y = player.grid_y - 32
        end
    elseif key == "down" then
        if testMap(0, 1) then
            player.grid_y = player.grid_y + 32
        end
    elseif key == "left" then
        if testMap(-1, 0) then
            player.grid_x = player.grid_x - 32
        end
    elseif key == "right" then
        if testMap(1, 0) then
            player.grid_x = player.grid_x + 32
        end
    end
end
```

You may have noticed that if you try to run this now, the program crashes. This is because our player starts outside of our map. If we expand the map we can fix the bug. Here is the full code with the expanded map.

```
function love.load()
    player = {
        grid_x = 256,
        grid_y = 256,
        act_x = 200,
        act_y = 200,
        speed = 10
    }
    map = {
        { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 },
        { 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1 },
        { 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1 },
        { 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1 },
        { 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1 },
        { 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1 },
        { 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 },
        { 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 },
        { 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 },
        { 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1 },
        { 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1 },
        { 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1 },
        { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 }
    }
```

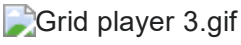
```
end

function love.update(dt)
    player.act_y = player.act_y - ((player.act_y - player.grid_y) * player.speed * dt)
    player.act_x = player.act_x - ((player.act_x - player.grid_x) * player.speed * dt)
end

function love.draw()
    love.graphics.rectangle("fill", player.act_x, player.act_y, 32, 32)
    for y=1, #map do
        for x=1, #map[y] do
            if map[y][x] == 1 then
                love.graphics.rectangle("line", x * 32, y * 32, 32, 32)
            end
        end
    end
end

function love.keypressed(key)
    if key == "up" then
        if testMap(0, -1) then
            player.grid_y = player.grid_y - 32
        end
    elseif key == "down" then
        if testMap(0, 1) then
            player.grid_y = player.grid_y + 32
        end
    elseif key == "left" then
        if testMap(-1, 0) then
            player.grid_x = player.grid_x - 32
        end
    elseif key == "right" then
        if testMap(1, 0) then
            player.grid_x = player.grid_x + 32
        end
    end
end

function testMap(x, y)
    if map[(player.grid_y / 32) + y][(player.grid_x / 32) + x] == 1 then
        return false
    end
    return true
end
```


Grid player 3.gif

There are several ways this could be improved.

- The art could be touched up, of course, using the techniques covered in this article: [1] (http://tommybrunn.com/2011/01/how-to-make-placeholder-art/)
- We could also implement some extra controls, such as binding the Escape key to love.event.push('quit'), which would quit the game.
- The player is inside of a closed map, but if the map was open and the player tried to leave it, the game would crash. We could easily add some checks to prevent this from happening.
- The map starts off at 32,32 when it would make more sense for it to start at 0,0. A little bit of arithmetic could fix this pretty easily, but it does make the code a bit more complex and hard to follow.

But for the purposes of this tutorial we have enough already.

Tutorial:Hamster Ball

Tutorial:Hamster Ball · revision 17113 · last edited 2023-07-05

This game is a very simple example of how to add a resource (in this case a simple image) to the engine and then move that around with the arrow keys. Here is the entire source, in case you haven't downloaded it:



Screenshot of the hamster ball.

```
-- Tutorial 1: Hamster Ball
-- Add an image to the game and move it around using
-- the arrow keys.
-- compatible with löve 0.6.0 and up

function love.load()
   hamster = love.graphics.newImage("hamster.png")
   x = 50
   y = 50
   speed = 300
end

function love.update(dt)
   if love.keyboard.isDown("right") then
      x = x + (speed * dt)
   end
   if love.keyboard.isDown("left") then
      x = x - (speed * dt)
   end

   if love.keyboard.isDown("down") then
      y = y + (speed * dt)
   end
   if love.keyboard.isDown("up") then
      y = y - (speed * dt)
   end
end

function love.draw()
   love.graphics.draw(hamster, x, y)
end
```



Hamster Ball Image (Note: The tutorial assumes you name this hamster.png)

It's not the most complicated program in the world, but we have to start somewhere. Keep in mind that everything after the `--` characters in the code is a comment and will be ignored by the program.

```
-- You can write whatever you want here!
```

Alright, next you need to know that for things to work you need to have a function called load. This is where the game is initiated and where everything begins. Although they aren't strictly required, it's best to also include `love.update()` and `love.draw()` for now. The meaning of the various functions will be described shortly.

```
-- This is an empty program
function love.load()
end

function love.update(dt)
end
```

```
function love.draw()
end
```

Alright, let's get down to explaining what the specific code for this game does.

The load function looks like this:

```
function love.load()
   hamster = love.graphics.newImage("hamster.png")
   x = 50
   y = 50
   speed = 300
end
```

What we are doing here is adding the file `hamster.png` to our game assigning that image to a variable called `hamster`. We are also declaring three variables: `x`, `y` and `speed`. These will all play a part in the program later.

The update function looks like this:

```
function love.update(dt)
   if love.keyboard.isDown("right") then
      x = x + (speed * dt)
   end
   if love.keyboard.isDown("left") then
      x = x - (speed * dt)
   end

   if love.keyboard.isDown("down") then
      y = y + (speed * dt)
   end
   if love.keyboard.isDown("up") then
      y = y - (speed * dt)
   end
end
```

First of all you will see that this function accepts a parameter called `dt` (an abbreviation of "delta-time") which represents the amount of time which has passed since this function was last called. It is in seconds, but because of the speed of todays processors is usually smaller than 1 (values like 0.01 are common). We are using this value to move the hamster ball at the same speed, no matter how fast the player's computer is.

In the function we check whether the keys right, left, up or down are pressed by calling the `love.keyboard.isDown(key)` function with the appropriate parameters (a full list of the keyboard keys can be found here). If any of the right keys are pressed then the `x` or `y` values will be incremented by the appropriate amount (in this case `speed` times `dt` which means that the ball will move at a speed of 300 pixels per second; try changing the `speed` value in the load function to see what happens).

The draw function looks like this:

```
function love.draw()
   love.graphics.draw(hamster, x, y)
end
```

Not a lot happening, but this is where the magic is done. The image contained in the `hamster` variable is drawn at the `x` and `y` coordinate.

That is basically it for your first game. Try messing around with the variables in the `love.load` function to see what effects they have.

Remember that .love files can be opened in standard archiving software, like WinZip or 7zip. They are actually just renamed .zip-files!

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Introduction to Shaders

Tutorial:Introduction to Shaders · revision 14927 · last edited 2023-07-05

This is a simple and basic tutorial explaining the basics of shaders. Here is a small example:

```
function love.load()
    shader = love.graphics.newShader("shader.fs")
end

function love.draw()
    love.graphics.setShader(shader)
    love.graphics.rectangle("fill", 0, 0, 100, 100)
    love.graphics.setShader()
end
```

The most important thing to remember when writing your shader is that the entry point is not the same as in pure GL. It's modified in LOVE.

```
vec4 effect( vec4 color, Image texture, vec2 texture_coords, vec2 screen_coords )
```

So instead of setting gl_FragCoord, you'd return the vec4 for the pixel color. Your texture coordinates are supplied for you, as well as the current texture, the system-wide current color setting, and the screen coordinate position of the current pixel. In pure GL, you'd supply your own uniform values for that stuff.

You can also send data to the shader in the form of uniform variables, like I mentioned above:

```
shader:send(name, value[s])
```

The value can be a number or a table of numbers for a vec2/3/4. Or more numbers for a matrix. Full list here.

If you're looking for a big repository of shaders already written for LOVE, you're out of luck as far as I know. There is, however, a "share a shader" (http://love2d.org/forums/viewtopic.php?f=4&t=3733) thread. That's the closest you can get right now.

If you don't know how shaders work, you can think of them as a small program that the graphics processor runs for **each pixel**. When you draw a rectangle, OpenGL transforms those coordinates into pixels on the screen (rasterization) and then runs those pixels through your shader. It's a very fast and efficient way to make modifications to images on the fly, and modern photorealistic or otherwise graphics would be nearly impossible (extremely impractical) without them.

The effect function in your shader returns a vec4 that represents the color you want to choose for the current pixel. There are a many functions built-in for math and for reading from textures. Here's a sample effect that'd simply draw a black and white texture.

```
vec4 effect( vec4 color, Image texture, vec2 texture_coords, vec2 screen_coords )
{
    vec4 c = Texel(texture, texture_coords); // This reads a color from our texture at the coordinates LOVE gave us (0-1, 0-1)
    return vec4(vec3(1.0, 1.0, 1.0) * (max(c.r, max(c.g, c.b))), 1.0); // This just returns a white color that's modulated by the brightest color channel at the given pixel in the texture. Nothing too complex, and not exactly the prettiest way to do B&W :P
}
```

-- Source: [1] (http://www.love2d.org/forums/viewtopic.php?f=3&t=32442)

See Also

- Shader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Isometric Graphics

Tutorial:Isometric Graphics · revision 14929 · last edited 2023-07-05

Isometric graphics are quite popular in RPGs and strategy games, so learning how to draw a grid of isometric cubes could be useful. For the sake of this tutorial I will assume that you are using cubes of equal size as we are working on making a grid of the cubes.

First, you need some source images. If you take a look at the source we have two wonderful cubes that are supposed to represent grass and dirt (using vaguely green and brown colors respectfully). For the sake of reference I have included a border around the cubes which will show up as an actual grid as it makes it easier to discern where one cube ends and the other begins. You should also make note at the fact that the bottom and right borders is missing, this is because the cubes are supposed to form a grid nicely and we don't want to get a double-border anywhere. In your own cubes you can probably draw whatever you want to connect the cubes as you don't HAVE to have a border around it, but I will do it for this example.

Note: This code is made for cubes, but works also if you want to create a grid with only the top face. This will result in the two values block_height and block_depth (as seen below) being the same.

First of all, once we have added the images to our game, we need some values. The width and the height of each cube (easily taken from the width and height of our source images) and a third value: the block_depth. This just means the height of the top face (easier to call it "depth" than to have to write "block_face_height" or something each time) and, in this case, is half the height of the cube itself:

```
block_width = grass:getWidth()
block_height = grass:getHeight()
block_depth = block_height / 2
```

These values will be used in doing the lovely math below.

Next, we need the grid itself. A simple 2-dimensional array will do, which we will fill with the value 1 (which represents a grass cube). I also specify a grid size so I can change things easily (if I want to):

```
grid_size = 20
grid = {}
for x = 1,grid_size do
   grid[x] = {}
   for y = 1,grid_size do
      grid[x][y] = 1
   end
end
```

For good measure (and to show where the various points would be) let's change two of the grid positions into the value 2 (which represents a dirt cube):

```
grid[2][4] = 2
grid[6][5] = 2
```

The actual drawing of the grid is like this (grid_x and grid_y are variables I declared earlier to keep a track of where the center of the grid should be; these would be used when moving the grid):

```
for x = 1,grid_size do
   for y = 1,grid_size do
      if grid[x][y] == 1 then
         love.graphics.draw(grass,
            grid_x + ((y-x) * (block_width / 2)),
```

```
            grid_y + ((x+y) * (block_depth / 2)) - (block_depth * (grid_size / 2)))
      else -- grid[x][y] == 2
         love.graphics.draw(dirt,
            grid_x + ((y-x) * (block_width / 2)),
            grid_y + ((x+y) * (block_depth / 2)) - (block_depth * (grid_size / 2)))
      end
   end
end
```

But I will focus on just the grass part, to make things easier:

```
love.graphics.draw(grass,
   grid_x + ((y-x) * (block_width / 2)),
   grid_y + ((x+y) * (block_depth / 2)) - (block_depth * (grid_size / 2)))
```

This draws the grass cubes starting from the top and moving down along the top-right side, then moves one step to the bottom-left and does it again. This ensures that the cubes are all drawn in the right order so that the ones that are in the "back" will not overlap the ones in the front. This part of the code ensures that the grid_x and grid_y variables point to the center of the grid:

```
- (block_depth * (grid_size / 2))
```

Note:If you don't like that the x-coordinate goes from the top to the right and the y-coordinate goes from the top to the left, you can easily switch them by switching their places in the line:

```
grid_x + ((y-x) * (block_width / 2)),
```

Which would result in this line:

```
grid_x + ((x-y) * (block_width / 2)),
```

Now what about adding a 3rd dimension? This is easy by adding the following code to the end of the y-coordinate you pass to *love.graphics.draw*:

```
- block_depth
```

(multiplied by every subsequent level you go up) So if we wanted to add another level to our grid you would simply put this into the nested for-loops:

```
love.graphics.draw(grass,
   grid_x + ((y-x) * (block_width / 2)),
   grid_y + ((x+y) * (block_depth / 2)) - (block_depth * (grid_size / 2)) - block_depth)
```

I hope this has taught you how to draw a simple isometric grid which you can use for fun and profit (if you ever make a profit remember me ^^).

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Networking with UDP

Tutorial:Networking with UDP · revision 28977 · last edited 2023-10-05

On this page

- The Client
  - love.load
  - love.update
  - love.draw
- The Server
- Conclusion
- See also
- Other languages

This is an introduction to networking, using *Luasocket*. Don't run away! Luasocket is compiled into LÖVE, and is really not that bad once you get used to it.

This tutorial assumes that you are familiar with Callbacks, and Lua in general. Networking should be considered a moderately advanced topic.

There are two basic kinds of sockets, and we'll be covering UDP in this tutorial. UDP networking is message-oriented (as opposed to TCP being stream-oriented), meaning that it's oriented around distinct (and otherwise independent) messages called *datagrams*.

In the long run it pays to have a solid understanding on how networking works, but for now lets just get cracking. :3 We'll start with the LÖVE client, then follow up with a stand-alone server written in Lua.

The Client

To start, we need to require the "socket" lib. *socket* provides low-level networking features.

```
local socket = require "socket"

-- the address and port of the server
local address, port = "localhost", 12345

local entity -- entity is what we'll be controlling
local updaterate = 0.1 -- how long to wait, in seconds, before requesting an update

local world = {} -- the empty world-state
local t
```

love.load

First up, we need a UDP socket, from which we'll do all our networking.

```
function love.load()
    udp = socket.udp()
```

Normally socket reads block (cause your game to stop and wait) until they have data. That doesn't suit us, so we tell it not to do that by setting the *timeout* to zero.

```
    udp:settimeout(0)
```

Unlike the server, we'll just be talking to the one machine, so we'll "connect" this socket to the server's address and port using *udp:setpeername*.



UDP is actually connectionless, this is purely a convenience provided by the socket library. It doesn't actually change the 'bits on the wire', and in fact we can change/remove this at any

time.

```
    udp:setpeername(address, port)
```

Seed the PRNG ↗, so we don't just get the same numbers each time. *entity* will be what we'll be controlling, for the sake of this tutorial. It's just a number, but it'll do. We'll just use *math.random* to give us a reasonably unique identity for little effort.


A random number to identify yourself isn't actually a very good way of doing this, but the "correct" ways are beyond the scope of this article.)

```
    math.randomseed(os.time())
    entity = tostring(math.random(99999))
```

Here we do our first bit of actual networking: we set up a string containing the data we want to send (using *string.format*) and then send it using *udp.send*. Since we used *udp:setpeername* earlier we don't even have to specify where to send it.

Thats... it, really. The rest of this is just putting this context and practical use.

```
    local dg = string.format("%s %s %d %d", entity, 'at', 320, 240)
    udp:send(dg) -- the magic line in question.

    -- t is just a variable we use to help us with the update rate in love.update.
    t = 0 -- (re)set t to 0
end
```

love.update

We start with a little bit of nonsense involving *t* we declared earlier; It's **very easy** to completely saturate a network connection if you aren't careful with the packets we send (or request!), so we hedge our chances by limiting how often we send (and request) updates.

(For the record, ten times a second is considered good for most normal games (including many MMOs ↗), and you shouldn't ever really need more than 30 updates a second, even for fast-paced games.)

We could send updates for every little move, but we'll consolidate the last update-worth here into a single packet, drastically reducing our bandwidth use.

```
function love.update(deltatime)
    t = t + deltatime -- increase t by the deltatime

    if t > updaterate then
        local x, y = 0, 0
        if love.keyboard.isDown('up') then  y=y-(20*t) end
        if love.keyboard.isDown('down') then    y=y+(20*t) end
        if love.keyboard.isDown('left') then    x=x-(20*t) end
        if love.keyboard.isDown('right') then   x=x+(20*t) end
```

Again, we prepare a packet *payload* using string.format, then send it on its way with udp:send. This is the move update mentioned above.

```
        local dg = string.format("%s %s %f %f", entity, 'move', x, y)
        udp:send(dg)
```

And again! This is a request that the server send us an update for the world state.



In most designs you don't request world-state updates, you just get them sent to you periodically.

There's various reasons for this, but there's one **BIG** one you will have to solemnly take note of: "anti-griefing". World-updates are probably one of biggest things the average game-server will pump out on a regular basis, and griefing with forged update requests would be simple effective. So they just don't support update requests, instead giving them out when they feel its appropriate

```
        local dg = string.format("%s %s $", entity, 'update')
        udp:send(dg)

        t=t-updaterate -- set t for the next round
    end
```

There could well be more than one message waiting for us, so we'll loop until we run out!

And here is something new, the much anticipated other end of `udp:send`! `udp:receive` will return a waiting packet (or nil, and an error message). *data* is a string, the **payload** of the far-end's *udp:send*. We can deal with it the same ways we could deal with any other string in Lua (needless to say, getting familiar with Lua's string handling functions (http://w w w.lua.org/manual/5.1/manual.html#5.4) is a must.)

```
    repeat
        data, msg = udp:receive()

        if data then -- you remember, right? that all values in lua evaluate as true, save nil and false?
```

`string.match` is our friend here, its part of *string.**, and *data* is (or should be!) a **string**. That funky set of characters bares some explanation, though. *(Which I haven't gotten to, but I'll leave you with a link to 5.4.1:Patterns (http://www.lua.org/manual/5.1/manual.html#5.4.1))*

```
            ent, cmd, parms = data:match("^(%S*) (%S*) (.*)")
            if cmd == 'at' then
                local x, y = parms:match("^(%-?[%d.e]*) (%-?[%d.e]*)$")
```

Confirming that the values you received are what you expect is important, since you never known who or what is on the other end (or in between...). Since this is just an example, we'll just use asserts.

And don't forget, even if you matched a "number", the result is still a **string**! Thankfully conversion is easy in Lua using `tonumber()`.

```
                assert(x and y)
                x, y = tonumber(x), tonumber(y)
                world[ent] = {x=x, y=y}
```

This case shouldn't trigger often, but its always a good idea to check (and log!) any unexpected messages and events. It can help you find bugs in your code...or people trying to hack the server. Never forget, you can not trust the client!

```
            else
                print("unrecognised command:", cmd)
            end
```

If *data* was `nil`, then *msg* will contain a short description of the problem (which are also double as error IDs...). The most common will be `'timeout'`, since we `socket:settimeout()` to zero, any time there isn't data *waiting* for us, it'll `'timeout'`. But we should check to see if its a *different* error, and act accordingly. In this case we don't even try to save ourselves, we just error out.

```
        elseif msg ~= 'timeout' then
            error("Network error: "..tostring(msg))
        end
    until not data
end
```

love.draw

Draw is stunningly simple, since its not really the meat of this example. It just loops over the *world* table, and print the name (key) of everything in there, at its own stored co-ords.

```
function love.draw()
    -- pretty simple, we
    for k, v in pairs(world) do
        love.graphics.print(k, v.x, v.y)
    end
end
```

And that's the end of the Client code.

The Server

The server is a little different, for starters its a stand-alone Lua program: it doesn't run in LÖVE.

Once again we begin by `require`ing socket, and creating a UDP socket.

(LuaSocket isn't *compiled into Lua by default. If you are on Windows just get the all-in-one [1]* *(https://github.com/rjpcomputing/luaforwindows/releases)* *installer, I wouldn't know for Mac, and Linux? you guys know what to do :3)*

```
local socket = require "socket"
local udp = socket.udp()
```

And once again, we set the `'timeout'` to zero.

But next we do something a little different; unlike the client, the server has to be specific about where its 'bound', or the poor clients will never find it. Thus while we can happily let the client auto-bind to whatever it likes, we have to tell the server to bind to something known.

The first part is which *interface* we should bind to, `'*'` basically means "all of them". *port* is simple, the system maintains a list of up to 65535 (!) "ports" ... really just numbers.

Point is that if you send to a particular port, then only things "listening" to that port will be able to receive it, and likewise you can only read data sent to ports you are listening too.

Generally speaking, if an address is which machine you want to talk to, then a port is what program on that machine you want to talk to.



On some operating systems, ports between *0* and *1024* are "reserved for privileged processes". Its a security precaution for those system. Generally speaking, just not using ports in that range avoids a lot of problems

```
udp:settimeout(0)
udp:setsockname('*', 12345)
```

We declare a whole bunch of local variables that we'll be using the in main server loop below. you probably recognise some of them from the client example, but you are also probably wondering what's with the fruity names, *msg_or_ip*? *port_or_nil*?

Well, we're using a slightly different function this time, you'll see when we get there.

```
local world = {} -- the empty world-state
local data, msg_or_ip, port_or_nil
local entity, cmd, parms
```

Indefinite loops are probably not something you are used to if you only know love, but they are quite common. And in fact love has one at its heart, you just don't see it. Regardless, we'll be needing one for our server. And this little variable lets us *stop* it :3

```
local running = true

print "Beginning server loop."
while running do
```

This next line looks familiar, I'm sure, but we're using `udp:receivefrom()` this time. its similar to receive, but returns the data, sender's ip address, and the sender's port (which you'll hopefully recognise as the two things we need to send messages to someone). We didn't have to do this in the client example because we just bound the socket to the server, but that also ignores messages from sources other than what we've bound to, which obviously won't do at all as a server.

(Strictly, we could have just used `udp:receivefrom()` (and its counterpart, `udp:sendto()`) in the client. there's nothing special about the functions to prevent it, however `send/receive` are convenient and perform slightly better.)

```
    data, msg_or_ip, port_or_nil = udp:receivefrom()
    if data then
        -- more of these funky match paterns!
        entity, cmd, parms = data:match("^(%S*) (%S*) (.*)")
```

The server implements a few more commands than the client does, the 'move' command updates the position of an entity *relative to its current position*, 'at' simply sets an entity's location (which we saw in the client), then there's *update*, which loops through the server's world-state and sends 'at' commands back to the client. and finally there's 'quit', which kills the server.

```
        if cmd == 'move' then
            local x, y = parms:match("^(%-?[%d.e]*) (%-?[%d.e]*)$")
            assert(x and y) -- validation is better, but asserts will serve.
            -- don't forget, even if you matched a "number", the result is still a string!
            -- thankfully conversion is easy in lua.
            x, y = tonumber(x), tonumber(y)
            -- and finally we stash it away
            local ent = world[entity] or {x=0, y=0}
            world[entity] = {x=ent.x+x, y=ent.y+y}
```

```
        elseif cmd == 'at' then
            local x, y = parms:match("^(%-?[%d.e]*) (%-?[%d.e]*)$")
            assert(x and y) -- validation is better, but asserts will serve.
            x, y = tonumber(x), tonumber(y)
            world[entity] = {x=x, y=y}
        elseif cmd == 'update' then
            for k, v in pairs(world) do
                udp:sendto(string.format("%s %s %d %d", k, 'at', v.x, v.y), msg_or_ip,  port_or_nil)
            end
        elseif cmd == 'quit' then
            running = false;
```

There's nothing much left to see, other than the *socket.sleep' call, which helps reduce the CPU load of the server.* Generally, people seem to prefer to just rely on blocking behaviour (which we turned off) to keep CPU usage down, but its nice to know how do this without blocking.

```
        else
            print("unrecognised command:", cmd)
        end
    elseif msg_or_ip ~= 'timeout' then
        error("Unknown network error: "..tostring(msg))
    end

    socket.sleep(0.01)
end

print "Thank you."
```

Conclusion

- The Complete Client Source
- The Complete Server Source

UDP ↗ is simple in its usage, but also relies on the developer to get a lot more right, since UDP doesn't make any assurances about the order that datagrams arrive, or even that they arrive at all. These are things that you obviously have to take into account when designing your protocol.

Additionally UDP datagrams have limited sizes; the luasocket Documentation specifically notes that it doesn't support anything over 8k, and frankly you should assume much less.

See also

- Networking with TCP
- luaSocket Reference (http://w 3.impa.br/~diego/softw are/luasocket/reference.html)
- Beej's Networking Tutorial (http://w w w .beej.us/guide/bgnet/)

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Networking with UDP-TheClient

Tutorial:Networking with UDP-TheClient · revision 8982 · last edited 2021-03-14

The complete source for the UDP client

```
-- to start with, we need to require the 'socket' lib (which is compiled
-- into love). socket provides low-level networking features.
local socket = require "socket"

-- the address and port of the server
local address, port = "localhost", 12345

local entity -- entity is what we'll be controlling
local updaterate = 0.1 -- how long to wait, in seconds, before requesting an update

local world = {} -- the empty world-state
local t

-- love.load, hopefully you are familiar with it from the callbacks tutorial
function love.load()

    -- first up, we need a udp socket, from which we'll do all
    -- out networking.
    udp = socket.udp()

    -- normally socket reads block until they have data, or a
    -- certain amout of time passes.
    -- that doesn't suit us, so we tell it not to do that by setting the
    -- 'timeout' to zero
    udp:settimeout(0)

    -- unlike the server, we'll just be talking to the one machine,
    -- so we'll "connect" this socket to the server's address and port
    -- using setpeername.
    --
    -- [NOTE: UDP is actually connectionless, this is purely a convenience
    -- provided by the socket library, it doesn't actually change the
    --'bits on the wire', and in-fact we can change / remove this at any time.]
    udp:setpeername(address, port)

    -- seed the random number generator, so we don't just get the
    -- same numbers each time.
    math.randomseed(os.time())

    -- entity will be what we'll be controlling, for the sake of this
    -- tutorial its just a number, but it'll do.
    -- we'll just use random to give us a reasonably unique identity for little effort.
    --
    -- [NOTE: random isn't actually a very good way of doing this, but the
    -- "correct" ways are beyond the scope of this article. the *simplest*
    -- is just an auto-count, they get a *lot* more fancy from there on in]

    entity = tostring(math.random(99999))

    -- Here we do our first bit of actual networking:
    -- we set up a string containing the data we want to send (using 'string.format')
    -- and then send it using 'udp.send'. since we used 'setpeername' earlier
```

```
    -- we don't even have to specify where to send it.
    --
    -- thats...it, really. the rest of this is just putting this context and practical use.
    local dg = string.format("%s %s %d %d", entity, 'at', 320, 240)
    udp:send(dg) -- the magic line in question.

    -- t is just a variable we use to help us with the update rate in love.update.
    t = 0 -- (re)set t to 0
end

-- love.update, hopefully you are familiar with it from the callbacks tutorial
function love.update(deltatime)

    t = t + deltatime -- increase t by the deltatime

    -- its *very easy* to completely saturate a network connection if you
    -- aren't careful with the packets we send (or request!), we hedge
    -- our chances by limiting how often we send (and request) updates.
    --
    -- for the record, ten times a second is considered good for most normal
    -- games (including many MMOs), and you shouldn't ever really need more
    -- than 30 updates a second, even for fast-paced games.
    if t > updaterate then
        -- we could send updates for every little move, but we consolidate
        -- the last update-worth here into a single packet, drastically reducing
        -- our bandwidth use.
        local x, y = 0, 0
        if love.keyboard.isDown('up') then  y=y-(20*t) end
        if love.keyboard.isDown('down') then    y=y+(20*t) end
        if love.keyboard.isDown('left') then    x=x-(20*t) end
        if love.keyboard.isDown('right') then   x=x+(20*t) end


        -- again, we prepare a packet *payload* using string.format,
        -- then send it on its way with udp:send
        -- this one is the move update mentioned above
        local dg = string.format("%s %s %f %f", entity, 'move', x, y)
        udp:send(dg)

        -- and again! this is a require that the server send us an update for
        --  the world state
        --
        -- [NOTE: in most designs you don't request world-state updates, you
        -- just get them sent to you periodically. theres various reasons for
        -- this, but theres one *BIG* one you will have to solemnly take note
        -- of: 'anti-griefing'. World-updates are probably one of biggest things
        -- the average game-server will pump out on a regular basis, and greifing
        -- with forged update requests would be simple effective. so they just
        -- don't support update requests, instead giving them out when they feel
        -- its appropriate]
        local dg = string.format("%s %s $", entity, 'update')
        udp:send(dg)

        t=t-updaterate -- set t for the next round
    end


    -- there could well be more than one message waiting for us, so we'll
    -- loop until we run out!
```

```
    repeat
        -- and here is something new, the much anticipated other end of udp:send!
        -- receive return a waiting packet (or nil, and an error message).
        -- data is a string, the payload of the far-end's send. we can deal with it
        -- the same ways we could deal with any other string in lua (needless to
        -- say, getting familiar with lua's string handling functions is a must.
        data, msg = udp:receive()

        if data then -- you remember, right? that all values in lua evaluate as true, save nil and false?

            -- match is our freind here, its part of string.*, and data is
            -- (or should be!) a string. that funky set of characters bares some
            -- explanation, though.
            -- (need summary of patterns, and link to section 5.4.1)
            ent, cmd, parms = data:match("^(%S*) (%S*) (.*)")
            if cmd == 'at' then
                -- more patterns, this time with sets, and more length selectors!
                local x, y = parms:match("^(%-?[%d.e]*) (%-?[%d.e]*)$")
                assert(x and y) -- validation is better, but asserts will serve.

                -- don't forget, even if you matched a "number", the result is still a string!
                -- thankfully conversion is easy in lua.
                x, y = tonumber(x), tonumber(y)
                -- and finally we stash it away
                world[ent] = {x=x, y=y}
            else
                -- this case shouldn't trigger often, but its always a good idea
                -- to check (and log!) any unexpected messages and events.
                -- it can help you find bugs in your code...or people trying to hack the server.
                -- never forget, you can not trust the client!
                print("unrecognised command:", cmd)
            end

        -- if data was nil, then msg will contain a short description of the
        -- problem (which are also error id...).
        -- the most common will be 'timeout', since we settimeout() to zero,
        -- anytime there isn't data *waiting* for us, it'll timeout.
        --
        -- but we should check to see if its a *different* error, and act accordingly.
        -- in this case we don't even try to save ourselves, we just error out.
        elseif msg ~= 'timeout' then
            error("Network error: "..tostring(msg))
        end
    until not data

end

-- love.draw, hopefully you are familiar with it from the callbacks tutorial
function love.draw()
    -- pretty simple, we just loop over the world table, and print the
    -- name (key) of everything in their, at its own stored co-ords.
    for k, v in pairs(world) do
        love.graphics.print(k, v.x, v.y)
    end
end

-- And thats the end of the udp client example.
```

Tutorial:Networking with UDP-TheServer

Tutorial:Networking with UDP-TheServer · revision 28025 · last edited 2022-09-30

The complete source for the UDP server

```
local socket = require "socket"

-- begin
local udp = socket.udp()

-- normally socket reads block until they have data, or a
-- certain amount of time passes.
-- that doesn't suit us, so we tell it not to do that by setting the
-- 'timeout' to zero
udp:settimeout(0)

-- unlike the client, the server has to be specific about where its
-- 'bound', or the poor clients will never find it.
-- thus while we can happily let the client auto-bind to whatever it likes,
-- we have to tell the server to bind to something known.
--
-- the first part is which "interface" we should bind to...a bit beyond this tutorial, but '*' basically means
"all of them"
-- port is simpler, the system maintains a list of up to 65535 (!) "ports"
-- ...really just numbers. point is that if you send to a particular port,
-- then only things "listening" to that port will be able to receive it,
-- and likewise you can only read data sent to ports you are listening too.
-- generally speaking, if an address is which machine you want to talk to, then a port is what program on that
machine you want to talk to.
--
-- [NOTE: on some operating systems, ports between 0 and 1024 are "reserved for
-- privileged processes". its a security precaution for those system.
-- generally speaking, just not using ports in that range avoids a lot of problems]
udp:setsockname('*', 12345)

local world = {} -- the empty world-state

-- We declare a whole bunch of local variables that we'll be using the in
-- main server loop below. you probably recognise some of them from the
--client example, but you are also probably wondering what's with the fruity
-- names, 'msg_or_ip'? 'port_or_nil'?
--
-- well, we're using a slightly different function this time, you'll see when we get there.
local data, msg_or_ip, port_or_nil
local entity, cmd, parms
-- indefinite loops are probably not something you used to if you only
-- know love, but they are quite common. and in fact love has one at its
-- heart, you just don't see it.
-- regardless, we'll be needing one for our server. and this little
-- variable lets us *stop* it :3
local running = true

-- the beginning of the loop proper...
print "Beginning server loop."
while running do
    -- this line looks familiar, I'm sure, but we're using 'receivefrom'
    -- this time. its similar to receive, but returns the data, sender's
```

```
    -- ip address, and the sender's port. (which you'll hopefully recognise
    -- as the two things we need to send messages to someone)
    -- we didn't have to do this in the client example because we just bound
    -- the socket to the server. ...but that also ignores messages from
    -- sources other than what we've bound to, which obviously won't do at
    -- all as a server.
    --
    -- [NOTE: strictly, we could have just used receivefrom (and its
    -- counterpart, sendto) in the client. there's nothing special about the
    -- functions to prevent it, indeed. send/receive are just convenience
    -- functions, sendto/receive from are the real workers.]
    data, msg_or_ip, port_or_nil = udp:receivefrom()
    if data then
        -- more of these funky match paterns!
        entity, cmd, parms = data:match("^(%S*) (%S*) (.*)")
        if cmd == 'move' then
            local x, y = parms:match("^(%-?[%d.e]*) (%-?[%d.e]*)$")
            assert(x and y) -- validation is better, but asserts will serve.
            -- don't forget, even if you matched a "number", the result is still a string!
            -- thankfully conversion is easy in lua.
            x, y = tonumber(x), tonumber(y)
            -- and finally we stash it away
            local ent = world[entity] or {x=0, y=0}
            world[entity] = {x=ent.x+x, y=ent.y+y}
        elseif cmd == 'at' then
            local x, y = parms:match("^(%-?[%d.e]*) (%-?[%d.e]*)$")
            assert(x and y) -- validation is better, but asserts will serve.
            x, y = tonumber(x), tonumber(y)
            world[entity] = {x=x, y=y}
        elseif cmd == 'update' then
            for k, v in pairs(world) do
                udp:sendto(string.format("%s %s %f %f", k, 'at', v.x, v.y), msg_or_ip,  port_or_nil)
            end
        elseif cmd == 'quit' then
            running = false;
        else
            print("unrecognised command:", cmd)
        end
    elseif msg_or_ip ~= 'timeout' then
        error("Unknown network error: "..tostring(msg))
    end

    socket.sleep(0.01)
end

print "Thank you."

-- and that the end of the udp server example.
```

Tutorial:Object Follows Another Object

Tutorial:Object Follows Another Object · revision 19649 · last edited 2021-03-14

Overview

In this tutorial, you'll be learning how to make an object follow another object.

Part 1: Creating the Objects
Part 2: Giving Movement
Part 3: The Whole Code

Part 1: Creating the Objects

To start off, you need to prepare the main.lua. First, we will create and render two objects, a red circle, and a green rectangle. These will be the objects used for this tutorial.

```
function love.load()
    circle = {}
    circle.x = 150          -- This will render the circle at 150px to the right along the x-axis.
    circle.y = 150          -- This will render the circle at 150px below along the y-axis.

    rect = {}
    rect.x = 300            -- This will render the rectangle at 300px to the right along the x-axis.
    rect.y = 150            -- This will render the rectangle at 150px below along the y-axis.
    rect.w = 100            -- This will render the rectangle with a width of 100px.
    rect.h = 100            -- This will render the rectangle with a height of 100px.
end

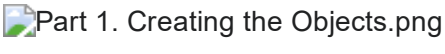
function love.update(dt)

end

function love.draw()
    -- This will render the red circle.
    love.graphics.setColor(255, 0, 0)
    love.graphics.circle("fill", circle.x, circle.y, 50)

    -- This will render the green rectangle.
    love.graphics.setColor(0, 255, 0)
    love.graphics.rectangle("fill", rect.x, rect.y, rect.w, rect.h)
end
```

You will now have something like this:

 Part 1. Creating the Objects.png

Part 2: Giving Movement

Next we will make the two objects move. One of them would be controlled by the player. The other one will follow the object controlled by the player. For the purpose of this tutorial, the player will be controlling the red circle. To do this, we will give the circle a speed value and add `love.keyboard.isDown` in the `love.update(dt)` function.

```
function love.load()
    ...
    -- Add below the circle axes.
    circle.speed = 500      -- This will give the circle a movement speed of 500.
end

function love.update(dt)
    -- These conditional statements add keyboard input.
    if love.keyboard.isDown("d") then               -- If player is holding down 'D',
    circle.x = circle.x + (circle.speed * dt)       -- Move circle to the right.
    end

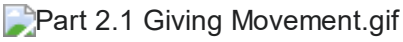
    if love.keyboard.isDown("a") then               -- If player is holding down 'A',
    circle.x = circle.x - (circle.speed * dt)       -- Move circle to the left.
    end

    if love.keyboard.isDown("w") then               -- If player is holding down 'W',
    circle.y = circle.y - (circle.speed * dt)       -- Move circle upwards.
    end

    if love.keyboard.isDown("s") then               -- If player is holding down 'S',
    circle.y = circle.y + (circle.speed * dt)       -- Move circle downwards.
    end
end

function love.draw()
    ...
end
```

You will now have something like this:



We aren't done yet though. We have only gave the red circle some movement. Next, we need to give the green rectangle movement and make it follow the red circle. To do that, we will give the green rectangle a speed value. Next we will check the location of the green rectangle in comparison to the red circle. If the right conditions are met, the green rectangle will move towards the red circle.

```
function love.load()
    ...
    -- Add below the rectangle values.
    rect.speed = 100        -- This will give the rectangle a movement speed of 100.
end

function love.update(dt)
    ...
    -- Add below the circle movement code.

    -- These will check the locations of the green rectangle in comparison to the red square.

    if rect.x < circle.x then                       -- If the rectangle is to the left of the circle:
    rect.x = rect.x + (rect.speed * 2.5 * dt)           -- Rectangle moves towards the right.
    end

    if rect.x > circle.x then                       -- If the rectangle is to the right of the circle:
    rect.x = rect.x - (rect.speed * 2.5 * dt)           -- Rectangle moves towards the left.
    end
```

```
    if rect.y < circle.y then                       -- If the rectangle is above the circle:
    rect.y = rect.y + (rect.speed * 2.5 * dt)           -- Rectangle moves downward.
    end

    if rect.y > circle.y then                       -- If the rectangle is below the circle:
    rect.y = rect.y - (rect.speed * 2.5 * dt)           -- Rectangle moves upward.
    end
end

function love.draw()
    ...
end
```

You will now have something like this:

 Part 2.2 Object Follows.gif

Congratulations! You've finished the tutorial!

Part 3: The Whole Code

```
function love.load()
    circle = {}
    circle.x = 150          -- This will render the circle at 150px to the right along the x-axis.
    circle.y = 150          -- This will render the circle at 150px below along the y-axis.
    circle.speed = 500      -- This will give the circle a movement speed of 500.

    rect = {}
    rect.x = 300            -- This will render the rectangle at 300px to the right along the x-axis.
    rect.y = 150            -- This will render the rectangle at 150px below along the y-axis.
    rect.w = 100            -- This will render the rectangle with a width of 100px.
    rect.h = 100            -- This will render the rectangle with a height of 100px.
    rect.speed = 100        -- This will give the rectangle a movement speed of 100.
end

function love.update(dt)
    if love.keyboard.isDown("d") then               -- If player is holding down 'D',
    circle.x = circle.x + (circle.speed * dt)       -- Move circle to the right.
    end

    if love.keyboard.isDown("a") then               -- If player is holding down 'A',
    circle.x = circle.x - (circle.speed * dt)       -- Move circle to the left.
    end

    if love.keyboard.isDown("w") then               -- If player is holding down 'W',
    circle.y = circle.y - (circle.speed * dt)       -- Move circle upwards.
    end

    if love.keyboard.isDown("s") then               -- If player is holding down 'S',
    circle.y = circle.y + (circle.speed * dt)       -- Move circle downwards.
    end

    if rect.x < circle.x then                   -- If the rect is to the left of the circle:
    rect.x = rect.x + (rect.speed * 2.5 * dt)       -- Rectangle moves towards the right.
    end

    if rect.x > circle.x then                   -- If the rect is to the right of the circle:
```

```
    rect.x = rect.x - (rect.speed * 2.5 * dt)       -- Rectangle moves towards the left.
    end

    if rect.y < circle.y then                   -- If the rect is above the circle:
    rect.y = rect.y + (rect.speed * 2.5 * dt)       -- Rectangle moves downward.
    end

    if rect.y > circle.y then                   -- If the rect is below the circle:
    rect.y = rect.y - (rect.speed * 2.5 * dt)       -- Rectangle moves upward.
    end
end

function love.draw()
    love.graphics.setColor(255, 0, 0)
    love.graphics.circle("fill", circle.x, circle.y, 50)

    love.graphics.setColor(0, 255, 0)
    love.graphics.rectangle("fill", rect.x, rect.y, rect.w, rect.h)
end
```

Tutorial:Object Follows Mouse

Tutorial:Object Follows Mouse · revision 29293 · last edited 2024-07-18

Overview

In this tutorial, you'll be learning how to make an object follow the mouse pointer inside the game's window.

Part 1: Detecting the Mouse
Part 2: The Object Follows
Part 3: The Whole Code

Part 1: Detecting the Mouse

So to start off, you need to prepare your main.lua. Now what we first need to do is to store the mouse's coordinates into two variables representing the x and y coordinate respectively. To do this, we will be using love.mouse.getPosition. For tutorial purposes, we will also be printing the coordinates to the screen so that we can see if the game is constantly updating the values in the variables.

```
mouse = {}

function love.load()

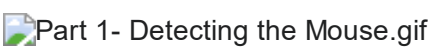
end

function love.update(dt)
    mouse.x, mouse.y = love.mouse.getPosition()  -- This gets the x and y coordinates of the mouse and assigns
those to these respectively.
end

function love.draw()
    love.graphics.setColor(1, 1, 1)
    love.graphics.print("Mouse Coordinates: " .. mouse.x .. ", " .. mouse.y)
end
```

> Note: We're putting the variables in the love.update function so that the game will constantly change the values within the variable. If you just declare it inside love.load and print it, it will always print the first mouse coordinates the game will detect and will never update the variables' values.

You will now have something like this:


Part 1- Detecting the Mouse.gif

Part 2: The Object Follows

Next we make the object follow. To do that, we will first create an object. For tutorial purposes, the object we will be creating is a red circle.

```
...
circle = {}

function love.load()
    circle.x = 300   -- This will render the circle at 300px to the right along the x-axis.
    circle.y = 300   -- This will render the circle at 300px below along the y-axis.
```

```
end

function love.update(dt)
    ...
end

function love.draw()
    -- Please draw the circle above the Mouse Coordinate text.
    love.graphics.setColor(1, 0, 0)                    -- Sets the circle's color to red.
    love.graphics.circle("fill", circle.x, circle.y, 50)  -- Draws a red circle at the given coordinates with a
radius of 50.
    ...
end
```

 Part 2.1- The Object Follows.PNG

Now that we have the red circle, we need to make it follow the mouse. To do this, we will be comparing the objects coordinate's to the mouse's current coordinates and if the conditions are met, then the object will move towards the mouse. The basics of character movement applies here. We need to declare the circle's movement speed and we must multiply this speed to the delta time.

```
...
circle = {}

function love.load()
    ...
    -- Add below the circle's coordinates
    circle.speed = 300  -- The circle's movement speed is 300.
end

function love.update(dt)
    ...
    -- Add below the love.mouse.getPosition()

    -- If the circle is to the left of the mouse:
    if circle.x < mouse.x then
    circle.x = circle.x + (circle.speed * 2.5 * dt)
    end

    -- If the circle is to the right of the mouse:
    if circle.x > mouse.x then
    circle.x = circle.x - (circle.speed * 2.5 * dt)
    end

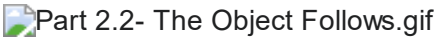
    -- If the circle is above the mouse:
    if circle.y < mouse.y then
    circle.y = circle.y + (circle.speed * 2.5 * dt)
    end

    -- If the circle is below the mouse:
    if circle.y > mouse.y then
    circle.y = circle.y - (circle.speed * 2.5 * dt)
    end
end

function love.draw()
    ...
end
```

> Note: The value of "2.5" there is an optional value. What this does is it increases the speed of the circle when it moves so that we don't have to keep on adjusting the movement speed that we set or so that we don't have to make any intricate operations on the delta time. You can change or remove completely if you want.

Now you have something like this:



Congratulations! You have successfully made an object follow the mouse.

Part 3: The Whole Code

```
mouse = {}
circle = {}

function love.load()
    circle.x = 300
    circle.y = 300

    circle.speed = 300
end

function love.update(dt)
    mouse.x, mouse.y = love.mouse.getPosition()

    if circle.x < mouse.x then
        circle.x = circle.x + (circle.speed * 2.5 * dt)
    end
    if circle.x > mouse.x then
        circle.x = circle.x - (circle.speed * 2.5 * dt)
    end
    if circle.y < mouse.y then
        circle.y = circle.y + (circle.speed * 2.5 * dt)
    end
    if circle.y > mouse.y then
        circle.y = circle.y - (circle.speed * 2.5 * dt)
    end
end

function love.draw()
    love.graphics.setColor(1, 0, 0)
    love.graphics.circle("fill", circle.x, circle.y, 50)

    love.graphics.setColor(1, 1, 1)
    love.graphics.print("Mouse Coordinates: " .. mouse.x .. ", " .. mouse.y)
end
```

Tutorial:Physics

Tutorial:Physics · revision 23633 · last edited 2023-07-05

In this example we will create a red ball that rolls around on a green ground.

The finished example is at the end of this page. All of these functions may be placed in one file: main.lua

We'll start in the love.load() function.

love.load()

First we need to set up a world for the physics bodies to exist in.

```
function love.load()
  love.physics.setMeter(64) --the height of a meter our worlds will be 64px
  world = love.physics.newWorld(0, 9.81*64, true) --create a world for the bodies to exist in with horizontal gravity of 0 and vertical gravity of 9.81
```

Now that a world has been created, we can add bodies, shapes, and fixture to it.

```
   objects = {} -- table to hold all our physical objects

  --let's create the ground
  objects.ground = {}
  objects.ground.body = love.physics.newBody(world, 650/2, 650-50/2) --remember, the shape (the rectangle we create next) anchors to the body from its center, so we have to move it to (650/2, 650-50/2)
  objects.ground.shape = love.physics.newRectangleShape(650, 50) --make a rectangle with a width of 650 and a height of 50
  objects.ground.fixture = love.physics.newFixture(objects.ground.body, objects.ground.shape) --attach shape to body

  --let's create a ball
  objects.ball = {}
  objects.ball.body = love.physics.newBody(world, 650/2, 650/2, "dynamic") --place the body in the center of the world and make it dynamic, so it can move around
  objects.ball.shape = love.physics.newCircleShape( 20) --the ball's shape has a radius of 20
  objects.ball.fixture = love.physics.newFixture(objects.ball.body, objects.ball.shape, 1) -- Attach fixture to body and give it a density of 1.
  objects.ball.fixture:setRestitution(0.9) --let the ball bounce

  --let's create a couple blocks to play around with
  objects.block1 = {}
  objects.block1.body = love.physics.newBody(world, 200, 550, "dynamic")
  objects.block1.shape = love.physics.newRectangleShape(0, 0, 50, 100)
  objects.block1.fixture = love.physics.newFixture(objects.block1.body, objects.block1.shape, 5) -- A higher density gives it more mass.

  objects.block2 = {}
  objects.block2.body = love.physics.newBody(world, 200, 400, "dynamic")
  objects.block2.shape = love.physics.newRectangleShape(0, 0, 100, 50)
  objects.block2.fixture = love.physics.newFixture(objects.block2.body, objects.block2.shape, 2)
```

Now to wrap up the love.load() function, let's set up the screen size and background color.

```
  --initial graphics setup
  love.graphics.setBackgroundColor(0.41, 0.53, 0.97) --set the background color to a nice blue
```

```
  love.window.setMode(650, 650) --set the window dimensions to 650 by 650 with no fullscreen, vsync on, and no antialiasing
end
```

Okay, that's enough for the initial set up of the physics engine. Now we need to edit the love.update() function.

love.update()

```
function love.update(dt)
  world:update(dt) --this puts the world into motion

  --here we are going to create some keyboard events
  if love.keyboard.isDown("right") then --press the right arrow key to push the ball to the right
    objects.ball.body:applyForce(400, 0)
  elseif love.keyboard.isDown("left") then --press the left arrow key to push the ball to the left
    objects.ball.body:applyForce(-400, 0)
  elseif love.keyboard.isDown("up") then --press the up arrow key to set the ball in the air
    objects.ball.body:setPosition(650/2, 650/2)
    objects.ball.body:setLinearVelocity(0, 0) --we must set the velocity to zero to prevent a potentially large velocity generated by the change in position
  end
end
```

Now that the world is updating, we can draw the ground and ball.

love.draw()

First, the ground.

```
function love.draw()
  love.graphics.setColor(0.28, 0.63, 0.05) -- set the drawing color to green for the ground
  love.graphics.polygon("fill", objects.ground.body:getWorldPoints(objects.ground.shape:getPoints())) -- draw a "filled in" polygon using the ground's coordinates
```

And finally, we can draw the circle that represents the ball and the blocks.

```
  love.graphics.setColor(0.76, 0.18, 0.05) --set the drawing color to red for the ball
  love.graphics.circle("fill", objects.ball.body:getX(), objects.ball.body:getY(), objects.ball.shape:getRadius())

  love.graphics.setColor(0.20, 0.20, 0.20) -- set the drawing color to grey for the blocks
  love.graphics.polygon("fill", objects.block1.body:getWorldPoints(objects.block1.shape:getPoints()))
  love.graphics.polygon("fill", objects.block2.body:getWorldPoints(objects.block2.shape:getPoints()))
end
```

There you have it! Put this file in a zip file, rename it to physics.love (or whatever), run it. And you'll have a ball rolling around in a lush green environment like I promised.

The main.lua

```
function love.load()
  -- the height of a meter our worlds will be 64px
  love.physics.setMeter(64)
```

```
  -- create a world for the bodies to exist in with
horizontal gravity
  -- of 0 and vertical gravity of 9.81
  world = love.physics.newWorld(0, 9.81*64, true)

  objects = {} -- table to hold all our physical objects

  -- let's create the ground
  objects.ground = {}
  -- remember, the shape (the rectangle we create next)
anchors to the
  -- body from its center, so we have to move it to (650/2,
650-50/2)
  objects.ground.body = love.physics.newBody(world, 650/2,
650-50/2)
  -- make a rectangle with a width of 650 and a height of
50
  objects.ground.shape =
love.physics.newRectangleShape(650, 50)
  -- attach shape to body
  objects.ground.fixture =
love.physics.newFixture(objects.ground.body,
                                                objects.ground.shape)

  -- let's create a ball
  objects.ball = {}
  -- place the body in the center of the world and make it dynamic, so
  -- it can move around
  objects.ball.body = love.physics.newBody(world, 650/2, 650/2, "dynamic")
  -- the ball's shape has a radius of 20
  objects.ball.shape = love.physics.newCircleShape(20)
  -- Attach fixture to body and give it a density of 1.
  objects.ball.fixture = love.physics.newFixture(objects.ball.body,
                                                 objects.ball.shape, 1)
  objects.ball.fixture:setRestitution(0.9) -- let the ball bounce

  -- let's create a couple blocks to play around with
  objects.block1 = {}
  objects.block1.body = love.physics.newBody(world, 200, 550, "dynamic")
  objects.block1.shape = love.physics.newRectangleShape(0, 0, 50, 100)
  -- A higher density gives it more mass.
  objects.block1.fixture = love.physics.newFixture(objects.block1.body,
                                                   objects.block1.shape, 5)

  objects.block2 = {}
  objects.block2.body = love.physics.newBody(world, 200, 400, "dynamic")
  objects.block2.shape = love.physics.newRectangleShape(0, 0, 100, 50)
  objects.block2.fixture = love.physics.newFixture(objects.block2.body,
                                                   objects.block2.shape, 2)

  -- initial graphics setup
  -- set the background color to a nice blue
  love.graphics.setBackgroundColor(0.41, 0.53, 0.97)
  love.window.setMode(650, 650) -- set the window dimensions to 650 by 650
end


function love.update(dt)
  world:update(dt) -- this puts the world into motion
```



Screenshot of the mostly finished product.

```
  -- here we are going to create some keyboard events
  -- press the right arrow key to push the ball to the right
  if love.keyboard.isDown("right") then
    objects.ball.body:applyForce(400, 0)
  -- press the left arrow key to push the ball to the left
  elseif love.keyboard.isDown("left") then
    objects.ball.body:applyForce(-400, 0)
  -- press the up arrow key to set the ball in the air
  elseif love.keyboard.isDown("up") then
    objects.ball.body:setPosition(650/2, 650/2)
    -- we must set the velocity to zero to prevent a potentially large
    -- velocity generated by the change in position
    objects.ball.body:setLinearVelocity(0, 0)
  end
end

function love.draw()
  -- set the drawing color to green for the ground
  love.graphics.setColor(0.28, 0.63, 0.05)
  -- draw a "filled in" polygon using the ground's coordinates
  love.graphics.polygon("fill", objects.ground.body:getWorldPoints(
                          objects.ground.shape:getPoints()))

  -- set the drawing color to red for the ball
  love.graphics.setColor(0.76, 0.18, 0.05)
  love.graphics.circle("fill", objects.ball.body:getX(),
                       objects.ball.body:getY(), objects.ball.shape:getRadius())

  -- set the drawing color to grey for the blocks
  love.graphics.setColor(0.20, 0.20, 0.20)
  love.graphics.polygon("fill", objects.block1.body:getWorldPoints(
                          objects.block1.shape:getPoints()))
  love.graphics.polygon("fill", objects.block2.body:getWorldPoints(
                          objects.block2.shape:getPoints()))
end
```

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:PhysicsCollisionCallbacks

Tutorial:PhysicsCollisionCallbacks · revision 28418 · last edited 2023-07-05

On this page

- Preface
- Tutorial
  - main.lua setup
  - World setup
  - World Callbacks
- Finished
  - Screenshots
  - main.lua
- See Also
- Other languages

Preface

If you do not have a good grasp on love.physics, you should first check out this physics tutorial.

In this tutorial we will create a collision between two objects that calls certain callbacks set by World:setCallbacks.


Making changes to a World is not allowed inside of the beginContact, endContact, preSolve, and postSolve callback functions, as BOX2D locks the world during these callbacks.

Tutorial

main.lua setup

Let's start by setting up main.lua with 3 love functions: love.load, love.update, and love.draw

```
function love.load()

end
function love.update(dt)

end
function love.draw()

end
```

World setup

Now that we have a framework setup, let's setup a physics world in love.load with newWorld. We will also use World:update.

```
function love.load()
    --Store the new world in a variable such as "world"
    world = love.physics.newWorld(0, 200, true)  --Gravity is being set to 0 in the x direction and 200 in the y direction.
end

function love.update(dt)
    --Never forget to update your world every frame.
```

```
    world:update(dt)
end
```

Now we want to create a ball and a ground. To do this we will need newBody, newCircleShape, and newRectangleShape. We'll connect the bodies to the shapes using newFixture.

```
function love.load()
    ... -- don't include this, it's just indicating where the existing code for the function should be

    ball = {}
        ball.b = love.physics.newBody(world, 400,200, "dynamic")  -- set x,y position (400,200) and let it move
and hit other objects ("dynamic")
        ball.b:setMass(10)                                        -- make it pretty light
        ball.s = love.physics.newCircleShape(50)                  -- give it a radius of 50
        ball.f = love.physics.newFixture(ball.b, ball.s)          -- connect body to shape
        ball.f:setRestitution(0.4)                                -- make it bouncy
        ball.f:setUserData("Ball")                                -- give it a name, which we'll access later
    static = {}
        static.b = love.physics.newBody(world, 400,400, "static") -- "static" makes it not move
        static.s = love.physics.newRectangleShape(200,50)         -- set size to 200,50 (x,y)
        static.f = love.physics.newFixture(static.b, static.s)
        static.f:setUserData("Block")
end
```

The objects are there now, but you can't yet see them. Let's draw them.

```
function love.draw()
    love.graphics.circle("line", ball.b:getX(),ball.b:getY(), ball.s:getRadius(), 20)
    love.graphics.polygon("line", static.b:getWorldPoints(static.s:getPoints()))
end
```

Now we should see a ball fall down and hit a solid rectangle.

World Callbacks

But what if we want more information on the two objects colliding? Now we will use World:setCallbacks to further dissect their collision(s).

First thing we do is set the world callbacks with World:setCallbacks. There are four callbacks for a collision: beginContact, endContact, preSolve, and postSolve.

> **beginContact** gets called when two fixtures start overlapping (two objects collide).
> **endContact** gets called when two fixtures stop overlapping (two objects disconnect).
> **preSolve** is called just before a frame is resolved for a current collision
> **postSolve** is called just after a frame is resolved for a current collision.

```
function love.load()
    ...  -- substitute for the rest of love.load

    world = love.physics.newWorld(0, 200, true)
        --These callback function names can be almost any you want:
        world:setCallbacks(beginContact, endContact, preSolve, postSolve)

    text       = ""   -- we'll use this to put info text on the screen later
    persisting = 0    -- we'll use this to store the state of repeated callback calls
```

Now define each function you just named.

```
function beginContact(a, b, coll)

end

function endContact(a, b, coll)

end

function preSolve(a, b, coll)

end

function postSolve(a, b, coll, normalimpulse, tangentimpulse)

end
```

These functions are called every time one of the collision actions happen. They pass in two fixtures and a collision object. The postsolve callback also contains the normal and tangent impulse for each collision contact point. These parameters can also be named to whatever you want. In this tutorial, we choose a, b, and coll.

- **a** is the first fixture object in the collision.
- **b** is the second fixture object in the collision.
- **coll** is the contact object created.
- **normalimpulse** is the amount of impulse applied along the normal of the first point of collision. It only applies to the postsolve callback, and we can ignore it for now.
- **tangentimpulse** is the amount of impulse applied along the tangent of the first point of collision. It only applies to the postsolve callback, and we can ignore it for now.

Say we want to print to screen whenever a callback is called. We just need to modify the **text** variable we added to love.load() earlier by appending a string every time a collision action happens. We need a bit of extra code to keep the output clean too.

A list of functions you can use on contacts can be found at the Contact page.

```
function love.update(dt)
    ... -- substitute for the rest of love.update

    if string.len(text) > 768 then    -- cleanup when 'text' gets too long
        text = ""
    end
end

function love.draw()
    ... -- substitute for the rest of love.draw

    love.graphics.print(text, 10, 10)
end

function beginContact(a, b, coll)
    x,y = coll:getNormal()
    text = text.."\n"..a:getUserData().." colliding with "..b:getUserData().." with a vector normal of: "..x..",
"..y
end
```

```
function endContact(a, b, coll)
    persisting = 0    -- reset since they're no longer touching
    text = text.."\n"..a:getUserData().." uncolliding with "..b:getUserData()
end

function preSolve(a, b, coll)
    if persisting == 0 then    -- only say when they first start touching
        text = text.."\n"..a:getUserData().." touching "..b:getUserData()
    elseif persisting < 20 then    -- then just start counting
        text = text.." "..persisting
    end
    persisting = persisting + 1    -- keep track of how many updates they've been touching for
end

function postSolve(a, b, coll, normalimpulse, tangentimpulse)
-- we won't do anything with this function
end
```

And now you know how to use world callbacks!

To better explore how this world behaves and see when the callbacks are invoked, add some controls to allow you to push around the ball:

```
function love.update(dt)
    world:update(dt)

    if love.keyboard.isDown("right") then
        ball.b:applyForce(1000, 0)
    elseif love.keyboard.isDown("left") then
        ball.b:applyForce(-1000, 0)
    end
    if love.keyboard.isDown("up") then
        ball.b:applyForce(0, -5000)
    elseif love.keyboard.isDown("down") then
        ball.b:applyForce(0, 1000)
    end

    if string.len(text) > 768 then    -- cleanup when 'text' gets too long
        text = ""
    end
end
```

Finished

Screenshots

main.lua

```
function love.load()
-- Create a physics World with gravity (0, 200) and enable sleep for objects at rest
    World = love.physics.newWorld(0, 100, true)
-- Set the collision callbacks for the World
    World:setCallbacks(beginContact, endContact, preSolve, postSolve)
```

```
    Platform = {}
-- Create a Platform body for the Platform
object at position (400, 400)
    Platform.body =
love.physics.newBody(World, 400, 400,
"static")
-- Create a rectangle shape for the
Platform object with width 200 and height
50
    Platform.shape =
love.physics.newRectangleShape(200, 50)
-- Create a fixture for the Platform
object using its body and shape
    Platform.fixture =
love.physics.newFixture(Platform.body,
Platform.shape)
-- Set the user data of the Platform
fixture to "Block"

Platform.fixture:setUserData("Platform")
```



Screenshot of the finished product.

```
    Ball = {}
-- Create a dynamic body for the Ball at position (400, 200)
    Ball.body = love.physics.newBody(World, 400, 200, "dynamic")
-- Set the mass of the Ball body to 10
    Ball.body:setMass(10)
-- Create a circle shape for the Ball with radius 50
    Ball.shape = love.physics.newCircleShape(50)
-- Create a fixture for the Ball using its body and shape
    Ball.fixture = love.physics.newFixture(Ball.body, Ball.shape)
-- Set the restitution (bounciness) of the Ball fixture to 0.4
    Ball.fixture:setRestitution(0.4)
-- Set the user data of the Ball fixture to "Ball"
    Ball.fixture:setUserData("Ball")

    Text = ""     -- we'll use this to put info Text on the screen later
    Persisting = 0 -- we'll use this to store the state of repeated callback calls

    love.window.setTitle ("Persisting: "..Persisting)
end

function love.update(dt)
-- Update the physics World with the specified time step (dt)
    World:update(dt)

-- Apply forces to the Ball based on keyboard input
    if love.keyboard.isDown("right") then
-- Apply a force of (1000, 0) to the Ball's body in the right direction
        Ball.body:applyForce(1000, 0)
    elseif love.keyboard.isDown("left") then
-- Apply a force of (-1000, 0) to the Ball's body in the left direction
        Ball.body:applyForce(-1000, 0)
    end
    if love.keyboard.isDown("up") then
-- Apply a force of (0, -5000) to the Ball's body in the upward direction
        Ball.body:applyForce(0, -5000)
    elseif love.keyboard.isDown("down") then
-- Apply a force of (0, 1000) to the Ball's body in the downward direction
```

```
        Ball.body:applyForce(0, 1000)
    end

    if string.len(Text) > 768 then-- Cleanup when 'Text' gets too long
        Text = "" -- Reset the Text variable when it exceeds the specified length
    end
end


function love.draw()
-- Draw the Ball as a circle using the Ball's position, radius, and line style
    love.graphics.circle("line", Ball.body:getX(), Ball.body:getY(), Ball.shape:getRadius(), 20)

-- Draw the Platform object as a polygon using the points of its shape and line style
    love.graphics.polygon("line", Platform.body:getWorldPoints(Platform.shape:getPoints()))

-- Draw the Text on the screen at position (10, 10)
    love.graphics.print(Text, 10, 10)
end



-- define beginContact, endContact, preSolve, postSolve functions:

function beginContact(a, b, coll)
    Persisting = 1
    local x, y = coll:getNormal()
    local textA = a:getUserData()
    local textB = b:getUserData()
-- Get the normal vector of the collision and concatenate it with the collision information
    Text = Text.."\n 1.)" .. textA.." colliding with "..textB.." with a vector normal of: ("..x..", "..y..")"
    love.window.setTitle ("Persisting: "..Persisting)
end

function endContact(a, b, coll)
    Persisting = 0
    local textA = a:getUserData()
    local textB = b:getUserData()
-- Update the Text to indicate that the objects are no longer colliding
    Text = Text.."\n 3.)" .. textA.." uncolliding with "..textB
    love.window.setTitle ("Persisting: "..Persisting)
end

function preSolve(a, b, coll)
    if Persisting == 1 then
    local textA = a:getUserData()
    local textB = b:getUserData()
-- If this is the first update where the objects are touching, add a message to the Text
        Text = Text.."\n 2.)" .. textA.." touching "..textB..": "..Persisting
    elseif Persisting <= 10 then
-- If the objects have been touching for less than 20 updates, add a count to the Text
        Text = Text.." "..Persisting
    end

-- Update the Persisting counter to keep track of how many updates the objects have been touching
    Persisting = Persisting + 1
    love.window.setTitle ("Persisting: "..Persisting)
end
```

```
function postSolve(a, b, coll, normalimpulse, tangentimpulse)
-- This function is empty, no actions are performed after the collision resolution
-- It can be used to gather additional information or perform post-collision calculations if needed
end
```

See Also

- love.physics
- World

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:PhysicsDrawing

Tutorial:PhysicsDrawing · revision 27941 · last edited 2022-07-26

This is not so much a tutorial but an explained code snippet that helps with physics debugging.

Drawing Physics Collisions

When developing with Box2D (aka love.physics) you will likely get to a point where the collisions don't quite match up with what you expect them to be due to some minor mistake.

At the end of this guide there's a complete snippet that will draw all collisions at the correct location. If you just want the code feel free to jump down and just copy & paste it. But we will go over what the code does bit by bit.

love.physics works via a few different sets of objects. Most notably. "World", "Body", "Shape" and "Fixture".

- "World" is the physics world. You can have multiple ones but only objects within the same world will collide.
- "Body" is the location of your object within the physics world. It defines which world this object belongs to, if it can move and the mass of the object (which will be calculated from the size of your shape by default). It also holds a reference to the fixture.
- "Shape" is the collision itself. If two shapes collide, the physics system will handle it. It also holds a reference to the fixture.
- "Fixture" is how we attach a shape to a body. It defines a few additional properties how collisions will be handled (such as friction, collision channels and a few more). It also holds a reference to the body and the shape.

In order to draw all shapes we need to start at the world. It handles all collisions. So it's gotta contain a list of things. Specifically. It contains a list of bodies which we can get in the following way:

```
listOfBodies = World:getBodyList()
```

We don't need a list of bodies though. We just wanna do something for every body. So we make a for loop out of it:

```
for _, body in pairs(World:getBodyList()) do
    -- ToDo
end
```

A body can have multiple shapes which are attached with multiple fixtures. So we have to iterate over the list of fixtures as well:

```
for _, body in pairs(World:getBodyList()) do
    for _, fixture in pairs(body:getFixtureList()) do
        -- ToDo
    end
end
```

A fixture however can only have one shape. So now we already have a reference to all collision fixtures (and therefore shapes) there are within this loop!

We can simply store a reference to the shape by doing

```
for _, body in pairs(World:getBodyList()) do
    for _, fixture in pairs(body:getFixtureList()) do
        local shape = fixture:getShape()

        -- ToDo Draw collision
    end
end
```

So next up is the drawing. There are multiple types of shapes. To be specific we have

- CircleShape
- PolygonShape
- EdgeShape
- ChainShape

Drawing circle shapes

So let's first look at the circle. We will draw it with the following code:

```
local cx, cy = body:getWorldPoints(shape:getPoint())
love.graphics.circle("fill", cx, cy, shape:getRadius())
```

cx & cy stand for "CircleX" and "CircleY". The position we want to draw it at.

body:getWorldPoints() will take any amount of points and convert them into world space (a point being two numbers. One for x and one for y). In other words. It will do "x + body:getX()", "y + body:getY()" for all points provided.

Then we simply draw the circle at the position we just calculated with the radius of the shape. I choose "fill" as first parameter. You can also use "line". "fill" means it'll be drawn as circle. "fill" will draw it as a ring.

Drawing a polygon shapes

Next up is the PolygonShape. A polygon is a list of points with up to 8 vertices (aka 16 numbers representing x and y. e.g. (x1,y1, x2,y2, ..., xn,yn))

We can abuse a little bit of functionality of Lua to make this a super small snippet of code. We can get all the points by calling "shape:getPoints()" (once we verified it's a polygon shape).

This will return up to 16 individual numbers. However. If we put this directly into another function without following parameter. It will simply put the next number as next parameter.

```
local function add(a, b)
    print(a + b)
end
local function f()
    return 1, 2
end

add(2, 2)   --> Prints 4.
add(f())    --> Prints 3. We returned multiple parameters which are simply considered as following parameters. So
a -> 1, b -> 2.
```

```
add(f(), 0) --> Prints 1. We returned multiple parameters. However those are directly followed by another value.
So only the first return value is used and the others will be discarded. a -> 1, b -> 0.
```

Knowing this we can do the following:

```
love.graphics.polygon("fill", body:getWorldPoints(shape:getPoints()))
```

Since there is no following parameter it will simply put as many points in the function as the shape has. We don't have to handle anything else!

Drawing edge and chain Shapes

Lastly we have edge and chain shapes. They are essentially the same. Internally they are handled a bit differently. But for our case we can assume an EdgeShape to be the same as a ChainShape with two points. A ChainShape can have two or more points (this is the only shape without a limit).

However. Both are made up of lines that may or may not loop around. And both have the function "shape:getPoints()" as well! So we do not have to treat them separately.

We pretty much do the same as we did for the polygon. But instead of drawing a polygon we draw a line.

```
love.graphics.line(body:getWorldPoints(shape:getPoints()))
```

Final code

This leaves us with the final code snippet that will draw all of our collisions at the correct location on the screen!

Happy debugging!

```
for _, body in pairs(world:getBodies()) do
    for _, fixture in pairs(body:getFixtures()) do
        local shape = fixture:getShape()

        if shape:typeOf("CircleShape") then
            local cx, cy = body:getWorldPoints(shape:getPoint())
            love.graphics.circle("fill", cx, cy, shape:getRadius())
        elseif shape:typeOf("PolygonShape") then
            love.graphics.polygon("fill", body:getWorldPoints(shape:getPoints()))
        else
            love.graphics.line(body:getWorldPoints(shape:getPoints()))
        end
    end
end
```

Tutorial:Tile-based Scrolling

Tutorial:Tile-based Scrolling · revision 9947 · last edited 2023-07-05

In this little tutorial we will cover a widely used technique used to draw game backgrounds: tilemaps. Using this technique, our backgrounds are going to be made up of pieces (tiles) that, drawn together, will allow us to create bigger and more complex levels without using huge amounts of memory. The basic idea is to have a 2d value table, where each value represents a tile. Using that table, we will know what tiles to draw at a certain position, given some basic parameters like the map width, height and the current coordinates of an imaginary viewport. Don't worry if you don't understand these terms right now, I'll explain them in more detail as we start coding. So, let's start with some variables:

```
-- our tiles
   tile = {}
   for i=0,3 do -- change 3 to the number of tile images minus 1.
      tile[i] = love.graphics.newImage( "tile"..i..".png" )
   end

   love.graphics.setNewFont(12)

   -- map variables
   map_w = 20
   map_h = 20
   map_x = 0
   map_y = 0
   map_offset_x = 30
   map_offset_y = 30
   map_display_w = 14
   map_display_h = 10
   tile_w = 48
   tile_h = 48
```

We first create a Lua table to store our tiles. As I already said, these are little bitmaps we'll use to draw our maps, something like mosaics, which are made up of smaller pieces. Since this will be a very simple tilemap, we'll just use four tiles, loading them inside a for loop. Now, the next thing we have to do is set up a handful of map and tile variables to be able to draw the map correctly. This ones should be pretty self-explanatory: besides height and width of the tiles and our tilemap, we also have the size of the region of tiles to display. The two offset variables are used to define the point where we will start drawing out tilemap. You might want to play a little with these variables to see their effects. Now, to the map structure itself:

```
map={
   { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
   { 0, 1, 0, 0, 2, 2, 2, 0, 3, 0, 3, 0, 1, 1, 1, 0, 0, 0, 0, 0},
   { 0, 1, 0, 0, 2, 0, 2, 0, 3, 0, 3, 0, 1, 0, 0, 0, 0, 0, 0, 0},
   { 0, 1, 1, 0, 2, 2, 2, 0, 0, 3, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0},
   { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0},
   { 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0},
   { 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
   { 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
   { 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
   { 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
   { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
   { 0, 2, 2, 2, 0, 3, 3, 3, 0, 1, 1, 1, 0, 2, 0, 0, 0, 0, 0, 0},
   { 0, 2, 0, 0, 0, 3, 0, 3, 0, 1, 0, 1, 0, 2, 0, 0, 0, 0, 0, 0},
   { 0, 2, 0, 0, 0, 3, 0, 3, 0, 1, 0, 1, 0, 2, 0, 0, 0, 0, 0, 0},
   { 0, 2, 2, 2, 0, 3, 3, 3, 0, 1, 1, 1, 0, 2, 2, 2, 0, 0, 0, 0},
   { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
   { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
```

```
   { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
   { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
   { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
}
```

As you can see, not much magic here, we just define a 2d table where each number represents an index of our tile table. In our example, the zero represents grass, the one represents dirt, the two represents stone and the three represents water. Obviously, your tiles may represent whatever you want, even parts of a bigger structure if you so desire. Let's continue:

```
function draw_map()
   for y=1, map_display_h do
      for x=1, map_display_w do
         love.graphics.draw(
            tile[map[y+map_y][x+map_x]],
            (x*tile_w)+map_offset_x,
            (y*tile_h)+map_offset_y )
      end
   end
end
```

To keep our `love.draw` callback clean, I decided to move our little map drawing routine its own function. Again, nothing really impressive here, we just go through our map table, checking every column (x values) for each row (y values). You can draw as many tiles as you want, just change the `map_display_h` and `map_display_w` variables accordingly. Just note that if you give them too high values you'll start drawing tiles off-screen, wasting time and resources. You can change the coordinates of the region to be drawn just by changing the `map_x` and `map_y` values.

Obviously, you have to call `draw_map()` in your `love.draw` callback. If you run your example right now, you should see our little tilemap on screen. You can't do any scrolling right now, but we'll get onto that right now, adding the following to our `love.keypressed` callback:

```
function love.keypressed(key, unicode)
   if key == 'up' then
      map_y = map_y-1
      if map_y < 0 then map_y = 0; end
   end
   if key == 'down' then
      map_y = map_y+1
      if map_y > map_h-map_display_h then map_y = map_h-map_display_h; end
   end

   if key == 'left' then
      map_x = math.max(map_x-1, 0)
   end
   if key == 'right' then
      map_x = math.min(map_x+1, map_w-map_display_w)
   end
end
```

This is fairly simple: first, we check if the arrow keys are being pressed and we update the `map_x` and `map_y` variables accordingly. Remember, here we're talking on tile coordinates so when we're increasing or decreasing `map_x` or `map_y` by one, we're actually moving 64 px (the size of a single tile). Finally, to avoid off-map scrolling, we include some boundary checking code. The *y*-coordinates shows a simple way of checking the boundaries while the boundary checks for *x*-coordinates use a more concise variant.

```
function love.draw()
  draw_map()
end
```

Finally, we use our draw_map () that we created earlier to actually display the tiles.

Now go and run your script. Isn't LÖVE cool? You bet it is, congratulations on making your first tilemap scroller! While this little example should be enough to get you up and running, there are quite a few possible enhancements:

- Writing our maps by hand is a little stupid, we should make them with tools like Tiled and load them in our game.
- Drawing characters inside the tilemap.
- Fine scrolling: As you probably noticed, the map scrolls tile by tile and you probably don't want that if you need to move your characters/object pixel by pixel. (Example)
- Multi-layered maps: Most real tile engines are able to render more than one layer. Together with fine scrolling, you would be able to scroll a near layer at a certain speed and move another one behind it more slowly.
- Character to map collisions.

See Also

Fine Tile-based Scrolling
Efficient Tile-based Scrolling

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tutorial:Using Input

Tutorial:Using Input · revision 24821 · last edited 2023-07-05

Capturing input events with LÖVE is really easy; in this tutorial we'll cover how to capture keyboard and mouse events using both object methods and callbacks. We'll start our tutorial by putting together an almost empty LÖVE program:

```
local text

function love.load()
   love.graphics.setNewFont(12)
   text = "Nothing yet"
end

function love.update(dt)

end

function love.draw()
   love.graphics.print( text, 330, 300 )
end
```

Capturing keyboard events

The easiest way to know if the user is pressing a key is calling the `love.keyboard.isDown` method, which has the following syntax:

```
love.keyboard.isDown( key )
```

The key parameter is a string representing the key we want to see if it's currently pressed. A simple example:

```
if love.keyboard.isDown( " " ) then
   text = "The SPACE key is held down!"
end
```

You can find the complete list of keys here. The best place to perform this check is inside the `love.update` callback: that way we're able to get input from the user and update our variables before drawing our stuff into the screen. So, our modified `love.update` callback should look like this:

```
function love.update(dt)
   if love.keyboard.isDown( " " ) then
      text = "The SPACE key is held down!"
   end
end
```

While this is fine and dandy if we only need to know which key or keys are currently pressed, we might also need to specify different behaviors when a certain key is pressed and/or released. An elegant way of doing this is using the keyboard callbacks `love.keypressed` and `love.keyreleased`. They work in a similar way of the already known `love.update` or `love.draw` callbacks, executing our code every time that event is triggered. For example:

```
function love.keypressed( key )
   if key == "return" then
      text = "RETURN has been pressed!"
```

```
   end
end

function love.keyreleased( key )
   if key == "return" then
      text = "RETURN has been released!"
   end
end
```

As you can see, these two callbacks will provide you a key variable which you can use to check if a given key has been pressed, released or both. Up to this point, our source file should look like this:

```
function love.load()
    love.graphics.setFont(12)
    text = "Nothing yet"
end

function love.update(dt)
   if love.keyboard.isDown( " " ) then
      text = "The SPACE key is held down!"
   end
end

function love.draw()
      love.graphics.print( text, 330, 300 )
end

function love.keypressed( key )
   if key == "return" then
      text = "RETURN is being pressed!"
   end
end

function love.keyreleased( key )
   if key == "return" then
      text = "RETURN has been released!"
   end
end
```

Capturing mouse events

So, we already know how to interact with our users through a keyboard. But what about that little rodent that sits on their desks? Well, mouse input works in a fairly similar way: we have a `love.mouse.isDown` method and the `love.mousepressed` and `love.mousereleased` callbacks. Let's add a few lines to our `love.update` callback:

```
if love.mouse.isDown("r") then
   text = "Right mouse button has been pressed"
end
```

As you can see, it's very similar to the `love.keyboard.isDown` and, again, you can see the full list of mouse-related parameters here. You can even check if the mouse wheel has been rolled up or down using this method. We also have two handy methods to know the current position of the mouse pointer inside our game window: `love.mouse.getX` and `love.mouse.getY`. Each one will return the current coordinate of the mouse pointer. Let's see an example by adding these line to the beginning of our `love.update` callback:

```
mouse_x = love.mouse.getX()
mouse_y = love.mouse.getY()
```

And this line to our `love.draw` callback:

```
love.graphics.print( "Mouse X: ".. mouse_x .. " Mouse Y: " .. mouse_y, 10, 20 )
```

The `love.mousepressed` and `love.mousereleased` callbacks work in a very similar way as their keyboard counterparts:

```
function love.mousepressed(x, y, button)
   if button == 1 then -- "Versions prior to 0.10.0 use the MouseConstant 'l'"
      text = "Left mouse button has been pressed"
   end
end

function love.mousereleased(x, y, button)
   if button == 1 then -- "Versions prior to 0.10.0 use the MouseConstant 'l'"
      text = "Left mouse button has been released"
   end
end
```

A cool feature of this callback is that you can know not only if a button has been pressed but also the position of the mouse pointer when the user pressed the button. This can be really useful if you need to put together some basic user interface elements like buttons or other objects that can interact with the mouse. A simple example:

```
function love.mousepressed(x, y, button)
   if button == 'l' then
      text = "Left mouse button has been pressed at X:"..x.." Y: "..y
   end
end
```

Finally, we have another two useful mouse-related methods: `love.mouse.setVisible` and `love.mouse.isVisible`. The first one will let you hide or show the mouse pointer and the second one will obviously let you know if the mouse pointer is visible or not. Let's add even more code to our `love.keypressed` callback:

```
if key == 'h' then
   if love.mouse.isVisible() then
      love.mouse.setVisible(false)
   else
      love.mouse.setVisible(true)
   end
   -- OR:
   -- love.mouse.setVisible(not love.mouse.isVisible())
end
```

In these few lines we check if the mouse pointer is visible or not and then we change its visibility: if it's visible we hide it and if it's already hidden we then show it. Fairly easy, isn't it?

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info